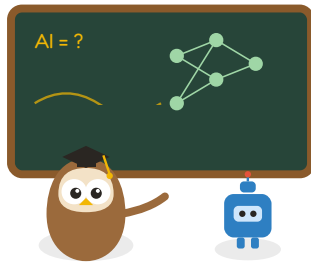


WHISKER WOODS ACADEMY PRESENTS

Learn AI Like a Kid

Stories, Animals & the Real Algorithms
Behind Artificial Intelligence



Professor Hoot & the Whisker Woods Class

A read-it-yourself adventure for curious minds, ages 10–14

WHISKER WOODS ACADEMY

Learn AI Like a Kid

Stories, Animals, and the Real Algorithms
Behind Artificial Intelligence

For curious minds, ages 10–14



Learn AI Like a Kid

Stories, Animals, and the Real Algorithms Behind Artificial Intelligence

First edition.

Text and illustrations created for the Whisker Woods Academy series.

All rights reserved. No part of this book may be reproduced in any form without written permission from the publisher, except for brief quotations used in reviews.

This book explains real artificial-intelligence ideas in a simplified, friendly way. The animals, dialogue, and stories are invented to make the ideas easy to picture. The algorithms, however, are real — the same families of methods used by scientists and engineers who build AI today.

The activities in this book are designed to be done with everyday materials and, for anything involving a computer or the internet, with a trusted adult nearby.

Printed for a 6 × 9 inch trade edition.

Set in Bitter, Fredoka, and Baloo 2.

*For every kid who has ever asked
“but how does it know that?”*

*You are already thinking like a scientist.
This book is for you.*

START HERE

How to Use This Book

Welcome to Whisker Woods Academy — a school in the middle of a forest where the students happen to be animals, and the subject they study happens to be the most exciting idea of our time: **artificial intelligence**, or AI.

Every chapter is a *class*. In each one, Professor Hoot the owl teaches one real AI idea, and the animals figure it out by living through a story. You don't need to be good at math. You don't need a computer. You just need to be curious — and you already are, or you wouldn't have opened this book.

To help you get the most out of each class, look out for these signs along the way:

How It Really Works

This is where we step out of the story and look at the actual algorithm — the real recipe that scientists and engineers use. This is the part that makes you smarter than most grown-ups about AI.

Try It Yourself

A hands-on activity, puzzle, or experiment you can do with paper, friends, or things around the house. Doing beats reading. Always try these.

Big Idea

The one sentence from the chapter that's worth remembering forever. If you only keep one thing, keep this.

Watch Out

A common mistake, a place where AI gets things wrong, or a trap that even real engineers fall into. Knowing these makes you wise.

Word Bank

The grown-up vocabulary for what you just learned — the exact words used by real scientists. Now they're yours, too. Every Word Bank term also lives in *The AI Dictionary* at the back of the book.

You can read the chapters in order, like a story, or hop straight to one that sounds exciting. Each class stands on its own. But the ideas *do* build on each other, like blocks in a tower — so if something feels confusing, flipping back a class or two usually helps.

Ready? Professor Hoot is already at the chalkboard. Let's meet the class.

THE STUDENTS

Meet the Class

Eight curious animals, one wise teacher, and a whole forest of ideas to figure out.



Professor Hoot

The wise old owl who runs the Academy. Asks more questions than he answers.



Bolt

A rabbit in a hurry. Wants the fast answer — and learns that fast isn't always smart.



Mango

A monkey who never stops asking “why?” Curious, clever, and a little mischievous.



Pip

A small, careful mouse. Shy, but notices the tiny details everyone else misses.



Tess

A tortoise who is never in a rush. Slow, steady, and almost always right in the end.



Coco

A parrot who repeats what she hears — which makes her perfect for learning about language.



Rex

A loyal dog who loves treats.
The best friend in the forest for
learning about rewards.



Ziggy

A zebra covered in patterns.
Loves sorting, grouping, and
spotting what makes things
different.

Table of Contents

PART ONE · MEET THE MACHINES THAT LEARN

- 1 What Is AI, Really?
- 2 How Machines Learn
- 3 Data: The Food That Feeds AI

PART TWO · SORTING, GROUPING & GUESSING

- 4 Decision Trees: The Question Game
- 5 Nearest Neighbors: Birds of a Feather
- 6 Clustering: Finding Your Flock
- 7 Linear Regression: The Line Into Tomorrow
- 8 Overfitting: Memorizing Isn't Learning

PART THREE · BRAINS MADE OF MATH

- 9 Neural Networks: A Brain of Tiny Switches
- 10 How a Network Learns: Practice Makes Better
- 11 Computer Vision: Teaching Machines to See
- 12 Memory & Sequences: Remembering What Came Before

PART FOUR · MACHINES THAT TALK & DECIDE

- 13 Language Models: How AI Learned to Talk
- 14 Attention: Focusing on What Matters
- 15 Reinforcement Learning: Training with Treats
- 16 Search & Pathfinding: Finding the Way Home

17 Recommendations: How AI Knows What You'll Like

PART FIVE · BIG IDEAS & BEING A GOOD MAKER

18 Generative AI: Machines That Imagine

19 Bias & Fairness: When AI Gets It Wrong

20 AI Safety: Keeping It Friendly

21 Your Turn: Becoming an AI Maker

PART ONE

Meet the Machines That Learn

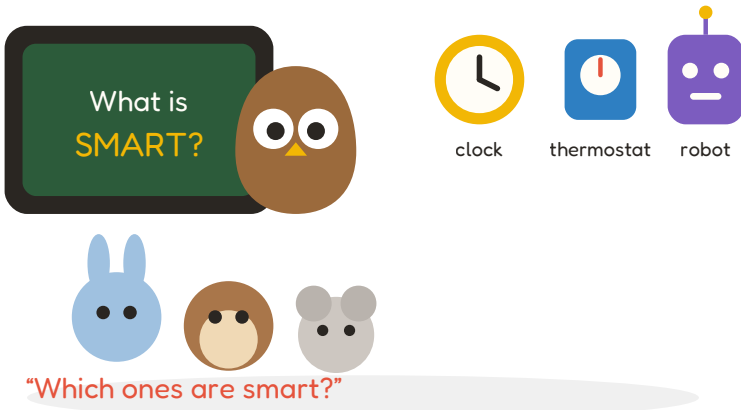
Before we can teach a machine anything, we have to understand one strange and wonderful idea: that a machine can learn at all. In these first three classes, we'll discover what AI really is, how learning from examples is different from following rules, and why everything starts with one humble ingredient – data.



CLASS ONE

What Is AI, Really?

First day of school — and the very first argument is about a question nobody can quite answer: what makes a machine “smart”?



On the very first morning at the Academy, before anyone had even found a seat, an argument broke out. It started — as the best arguments do — with a perfectly simple question. Professor Hoot had written one word on the chalkboard in big chalky letters: **SMART**. Then he tapped it with a wing and said, “Tell me, class. What makes a machine smart?”

That was all it took.

Bolt: *"The clock! The clock is smart. It always knows the time, even at night, even when I'm asleep. I never know the time!"*

Mango: *"A clock isn't smart, it just ticks. The thermostat is the smart one — it feels when the room gets cold and turns the heat on all by itself."*

Pip: *(very quietly) "My grandmother has a calculator. It can multiply huge numbers faster than anyone in this whole forest. Isn't that the smartest of all?"*

Professor Hoot let them argue. He rather enjoyed it. A clock, a thermostat, a calculator — three machines, and three animals each completely certain they'd found the smart one. When the noise finally died down, the old owl asked a question of his own, the way he always did.

Professor Hoot: *"Here is what I want to know. The clock, the thermostat, the calculator — does any of them ever do something its makers did not already tell it to do? Does any of them ever learn anything new?"*

The classroom went quiet. Pip's whiskers twitched. Mango scratched his head.

Mango: *"...No. The calculator does exactly the same thing every time. Two plus two is always four. It was built that way."*

Professor Hoot: *"Exactly so. These machines are clever — but they only ever follow rules that a person wrote for them, step by step. The thermostat has a rule: if the room is below 20 degrees, turn on the heat. That's a useful rule! But it is not what we mean by artificial intelligence."*

So what is AI?

Professor Hoot wiped the board clean and wrote two words: **Artificial Intelligence**. "Artificial," he said, "just means *made by people*. And

intelligence is the part we've been arguing about. We use the words **artificial intelligence** for a machine that does a job that normally seems to need a human mind — things like *recognizing* a face, *understanding* a sentence, *deciding* the best move in a game, or *predicting* tomorrow's weather."

Bolt: "But the thermostat decides to turn on the heat! That's deciding!"

Professor Hoot: "A fair point, Bolt. Here is the real secret, the thing that separates a thermostat from true AI." He drew a thick line down the middle of the board. "A thermostat follows a rule we gave it. The most exciting AI does something different — it learns the rule itself, by looking at examples. Nobody writes down 'this is what a cat looks like.' Instead, we show the machine thousands of cat photos, and it figures out the pattern on its own. That figuring-out is called machine learning, and it is the heart of everything we'll study this year."

Pip raised a trembling paw. "So a calculator follows rules... but a machine that looks at lots of examples and teaches *itself* — that one is doing AI?" Professor Hoot beamed. "Pip," he said, "you have understood the whole class in a single sentence."

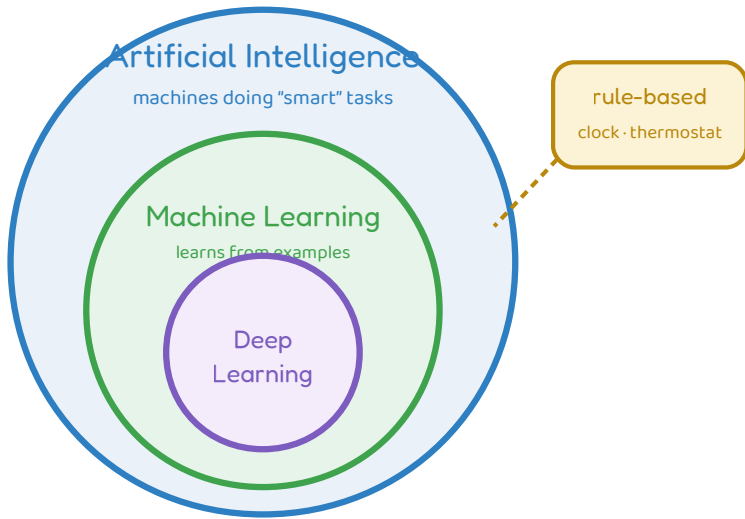
How It Really Works

Artificial intelligence is the big idea of making machines do tasks that *seem* to need human intelligence — recognizing, understanding, deciding, predicting. That's it. It's a goal, not a single gadget.

There are two very different ways to chase that goal. The old way is to write down **rules** by hand: *if this, then that*. A thermostat and a calculator work this way, and so do plenty of useful programs. They never get any smarter than the rules a person typed in.

The newer, more powerful way is **machine learning**: instead of writing the rules, you show the machine lots of *examples* and let it find the pattern by itself. Machine learning is one important *part* of AI — not the whole thing, but the part doing nearly all the amazing tricks you've heard about. Inside machine learning there's an even more powerful technique called **deep learning**, which we'll meet in a later class.

One more honest truth. Every real AI today is a **narrow AI**: it is good at exactly *one* kind of task. A program that plays chess like a champion cannot read you a bedtime story, and a program that recognizes cats has no idea what chess even is. The all-knowing, do-everything robot from the movies is called *general AI*, and despite what films pretend, **nobody has built one**. It may never exist.



AI is the whole idea. Machine learning lives inside it, and deep learning lives inside that. Plain rule-following gadgets sit off to the side — useful, but not learning.

“So we were all a little bit wrong,” said Mango, grinning. “The clock and the thermostat and the calculator are clever machines... they're just not learners.” Professor Hoot nodded. “And being a *learner*,” he said, “is what this whole Academy is about.”

Try It Yourself: The Smart-or-Not Hunt

Walk around your home and write down **five machines** you find — a microwave, a toaster, a TV remote, a phone, a video-game console, a smart speaker, anything. For each one, make two columns and decide:

1. Does it just **follow fixed rules** a person built in (like the thermostat), or does it seem to **learn from examples** and get better over time?
2. Write down *why* you think so. What clue gave it away?

Tricky ones to argue about: a phone that recognizes *your* face, a video app that keeps suggesting shows you'll like, a speaker that understands what you say. Those lean toward AI — because somewhere inside, a machine learned a pattern from lots of examples. A plain toaster? That's just rules. There are no wrong answers, only good reasons.

Watch Out

Movies love to show AI as a robot that is *alive* — one that feels lonely, falls in love, gets angry, or secretly plots to take over the world. That makes a great story, but it is **not** what real AI is. Today's AI does not think, want, or feel anything. When a chatbot writes "I'm happy to help!", it is matching patterns from text it was trained on, not actually feeling happy. It's a very fancy pattern machine — closer to the calculator than to a person. Being impressed by AI is smart; believing it's secretly alive is exactly the myth this book will help you see through.

Word Bank

Artificial Intelligence (AI)

The big idea of making machines do tasks that normally seem to need human intelligence — recognizing, understanding, deciding, or predicting.

Machine Learning

A part of AI where a machine learns the rules by itself from lots of examples, instead of being given the rules by a person.

Algorithm

A step-by-step recipe a computer follows to do a job. Some algorithms just follow fixed rules; learning algorithms figure out their rules from data.

Narrow AI

An AI that is good at one specific task and only that task. Every real AI today is narrow. The do-everything “general AI” of movies does not exist.

Model

What a machine-learning algorithm produces after studying examples — the finished pattern-spotter that does the actual recognizing or predicting.

Big Idea

A machine that follows rules is clever; a machine that *learns the rules from examples* is doing AI — and that's the magic we're here to learn.

Before You Turn the Page...

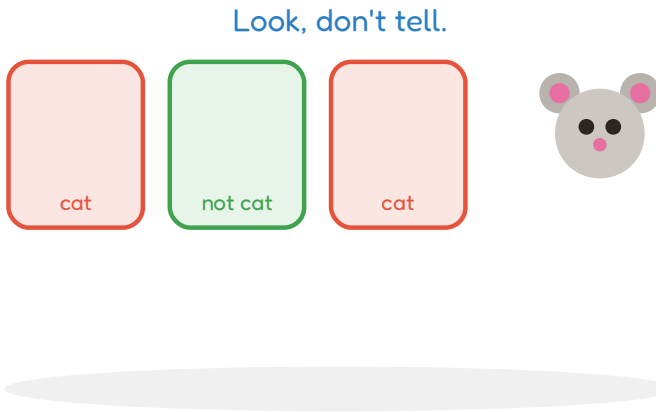
- **AI** means machines doing tasks that seem to need human intelligence — recognizing, deciding, understanding, predicting.
- A clock, thermostat, or calculator just **follows fixed rules**; the real magic is a machine that **learns from examples**.
- **Machine learning** is the learning part of AI; **deep learning** is a powerful piece inside that.
- Every real AI today is **narrow** — good at one thing — and it is not alive and has no feelings.

Next class: if machines learn from examples, how on earth do they actually do it? Time to find out How Machines Learn. →

CLASS TWO

How Machines Learn

Why we teach machines by showing them examples instead of telling them every single rule.



Professor Hoot tapped the chalkboard and beamed at the class.

P “Today,” he announced, “you are going to teach me something. Write down the rules — the *exact* rules — for recognizing a cat. When you're done, I'll follow them perfectly, like a machine. If your rules are good, I'll never be wrong.”

Bolt's paw shot up before Hoot had even finished. “Easy! Four legs, whiskers, and fur. Done.” He folded his arms, pleased with himself.

Professor Hoot: “*Four legs, whiskers, fur. Wonderful.*” He drew a wobbly dog on the board. “*But Rex has four legs, whiskers, and fur. By your rule, Rex is a cat.*”

The class giggled. Rex, who had been napping, lifted one ear at the sound of his name.

Pip the mouse, who almost never spoke, had been quietly studying the picture cards on her desk. She noticed something. “Professor... the rule breaks the other way too.” She held up a card of a cat curled into a sleeping ball. “This is a cat. But you can't see *any* of its four legs. They're tucked underneath.”

Professor Hoot: *“Ah. Pip notices the tiny things, as always. So our rule says ‘cat’ for a dog, and ‘not a cat’ for a sleeping cat. Let's try to patch it.”*

So they tried. *Pointy ears!* (Dogs have those.) *Says meow!* (You can't hear a photograph.) *Small and soft!* (So is a rabbit, and Bolt took mild offense.) Every single rule they wrote down was either too strict, too loose, or broke the moment someone found a sneaky exception. The board filled up with crossed-out rules until it looked like a tic-tac-toe disaster.

Pip: *“There are too many kinds of cats. Fat ones, hairless ones, kittens, cats in funny hats. We'd need a million rules — and we'd still miss one.”*

The better way: show, don't tell

Professor Hoot smiled the smile of an owl who had been waiting for exactly this moment. He swept the broken rules off the board and set down a tall stack of cards instead.

Professor Hoot: *“Then let's stop telling. Pip, don't try to write the rule. Just look. I'll show you hundreds of pictures. Each one I'll tell you: this is a cat... this is not a cat... cat... not a cat... Your only job is to watch and let your brain find the pattern by itself.”*

Pip looked. And looked. Card after card after card. At first she guessed randomly. But after a hundred examples, something clicked — not a rule she could say out loud, but a *feeling* for cat-ness she could now recognize instantly. Hoot held up a brand-new picture she had never seen. A cat in a raincoat. “Cat,” said Pip, without hesitation. She was right.

Pip: *“I didn't memorize that picture — I'd never seen it! I just... learned what cats look like from all the others.”*

Professor Hoot: *“And that, class, is how machines learn. Not from rules we type in, but from examples we show them.”*

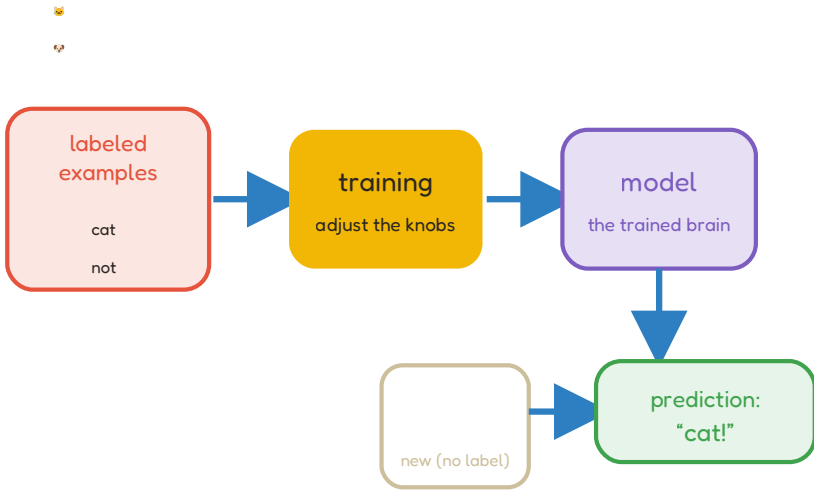
How It Really Works

What Pip did has a real name: **supervised learning**. The word *supervised* means each example comes with the right answer attached — a **label**. A photo labeled “cat.” A photo labeled “not cat.” The machine always gets to peek at the correct answer while it studies.

Inside the machine there's a thing called a **model** — think of it as a big adjustable guessing-machine full of knobs. At the start, every knob is set randomly, so the model guesses like Pip on her first card: pure luck. Then **training** begins. The machine looks at an example, makes a guess, and checks the label. Wrong? It nudges its knobs a little so that next time it would be closer to right. Right? Good — keep going. Do this across thousands of labeled examples and the knobs slowly settle into a pattern that *works*.

Here's the part people miss: the real goal is **not** to ace the examples it already studied. The goal is **generalization** — getting *new*, never-seen examples right, like the cat in the raincoat. A model that only knows its practice cards has memorized, not learned.

So there are two ways to make a computer do a job. The old way is **programming with rules**: a human writes down every step (“if four legs and whiskers, then cat”). The new way is **learning from data**: you don't write the rule at all — you show examples and let the model discover the pattern itself. For messy things like cats, faces, or voices, learning from data wins by a mile.



The whole pipeline: labeled examples → training → a model → a prediction on a brand-new example it has never seen.

Notice what the model never receives in that last step: a label.

N The new cat arrives bare, and the model must *decide*. That moment — guessing the answer for something fresh — is the whole reason we trained it in the first place.

Try It Yourself: Train Your Friend on “Blorps”

You're going to be Professor Hoot, and your friend is going to be the machine. The trick: **you must never tell them the rule.**

1. Secretly invent a category called a **blorp**. Pick a rule only you know — say, “a doodle is a blorp if it has exactly three dots inside it.” Don't write the rule down where your friend can see.
2. Draw about eight doodles. Label each one out loud: “*This is a blorp... this is NOT a blorp...*” Make some blorps and some non-blorps so the pattern is there to be found. These are your **training examples**.
3. Now the test. Draw five *new* doodles your friend has never seen and ask: “Blorp or not?” Don't tell them the answers yet.
4. Check their guesses. If they nailed the new ones, they **generalized** — they learned your rule from examples alone, exactly like a machine.

Play it again with a trickier rule (“a blorp has a curvy edge AND is colored in”) and watch how many more examples your friend needs to crack it.

Watch Out

The machine learns whatever pattern is actually in your examples — even one you didn't mean to teach. If every cat you show is orange, the model may quietly decide “cat = orange,” and then call a pumpkin a cat and a black cat a dog. And if you show too *few* examples, it grabs the first flimsy pattern it finds and sticks with it. Garbage examples in, garbage learning out. We'll dig into this in Class Three (where good data comes from) and again in the class on **overfitting**, where a model memorizes instead of generalizing.

Word Bank

Label

The correct answer attached to an example — “cat” or “not cat.” In supervised learning, every training example carries one.

Training

The practice phase: the model guesses, checks the label, and adjusts itself to be a little less wrong each time.

Model

The adjustable guessing-machine the computer builds. Training is what shapes it; afterward it's what makes the predictions.

Supervised learning

Learning from examples that already come with the right answers, so the machine can check its work as it goes.

Generalization

The real goal — getting new, unseen examples right, instead of just memorizing the ones you trained on.

Big Idea

We don't teach a machine by writing down every rule — we show it labeled examples and let it find the rule itself.

Before You Turn the Page...

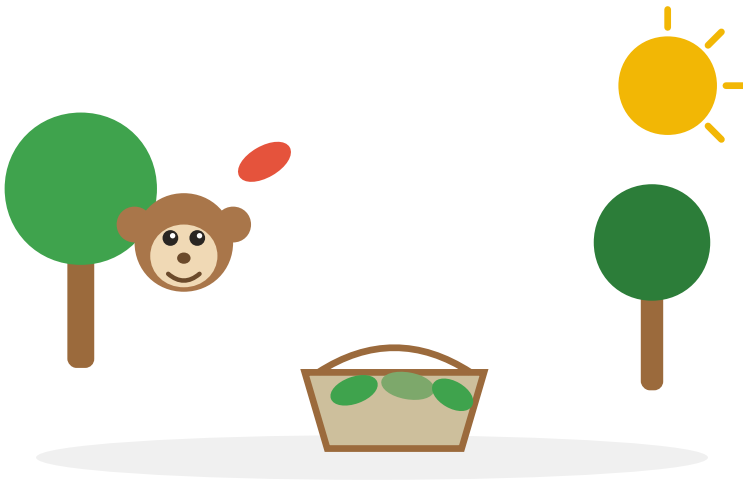
- Some things are too messy for rules — every “cat rule” you write down **breaks** on some sneaky exception.
- Instead, you show the machine many **labeled examples** and let it find the pattern — that's **supervised learning**.
- **Training** means nudging the **model** over and over until its guesses match the labels.
- The point isn't to memorize the examples — it's to **generalize** to new ones it's never seen.

Next class: a model is only as good as what you feed it. Meet Data: The Food That Feeds AI. →

CLASS THREE

Data: The Food That Feeds AI

An AI is only as good as what you feed it — so be very careful what goes in the basket.



Field trip day! Professor Hoot lined up the class at the edge of the forest, handed each student an empty basket, and announced the mission: “Today we are going to teach a machine to tell an **oak leaf** from a **maple leaf**. But a machine has never seen a leaf in its life. So first, we must bring it *examples* — lots and lots of leaves, each one labeled with its name. We call all those examples **data**.”

Mango: *“Examples? That's easy! I can collect a HUNDRED leaves before lunch!” And off he scampered, swinging through the branches.*

Mango was fast. But Mango was also a little careless. He grabbed only the *big* leaves, because big ones were easy to spot from up in the trees. He only collected on the sunny side of the path, because that's where the swinging was fun. And once or twice, in a hurry, he dropped a maple leaf into the “oak” pile without really checking.

Mango: *“Done! A whole basket! Best collector in the class!”*

Tess the tortoise had collected far fewer leaves — but she had done it differently. She took big leaves *and* small ones. She walked the shady side of the path as well as the sunny side. And for every single leaf, she stopped, looked carefully, and wrote the correct name on a little tag before dropping it in.

Professor Hoot: *“Mango, you brought us a lot of leaves. Tess, you brought us careful ones. Let's feed both baskets to the machine and see which one teaches it better.”*

Garbage in, garbage out

The machine studied Mango's basket first. Then Professor Hoot held up a small oak leaf from the shady part of the forest and asked the machine to name it. The machine thought hard and answered: “Maple!” Wrong. It tried another small leaf. “Maple!” Wrong again.

Mango: *“What?! I gave it a hundred leaves! Why is it so bad?”*

Professor Hoot: *“Because every leaf you gave it was big and from the sunny side — and a few were labeled wrong. The machine didn't learn what an oak leaf is. It learned that ‘oak means big and bright.’ Show it a*

small leaf in the shade, and it's lost." He ruffled his feathers. "There's an old saying for this, class. Garbage in, garbage out."

Then they fed it Tess's careful basket. Big leaves, small leaves, sunny ones, shady ones — every one correctly labeled. This time the machine named the small shady oak leaf right away. Fewer leaves, but *better* leaves, taught it far more.

How It Really Works

To a machine, every example is just a list of **features** — the measurable facts about a thing. A leaf's features might be its *size*, the shape of its *edges* (smooth, pointy, or wavy), and its *color*. Alongside the features we write the right answer, called the **label**: “oak” or “maple.” A whole pile of labeled examples is a **dataset**.

For the machine to learn the real pattern, that dataset has to be three things at once. It needs to be **big enough** — a handful of leaves won't reveal the rule. It needs to be **varied** — big and small, sunny and shady — so the machine sees the full range of what a leaf can be. And the labels need to be **correct**, because the machine trusts every label completely; one mislabeled leaf teaches it something false.

Here's the quiet danger Mango fell into. If your data only ever shows *one kind* of example, the machine becomes blind to the rest. Train it only on big leaves and it decides small leaves aren't really leaves. When that blindness ends up being unfair to certain things — or certain *people* — we call it **bias**. It's such a big deal that it gets its own class later; for now, just remember: lopsided data makes a lopsided machine.

One more trick the pros use: they don't feed the machine *all* the data. They hide some away. The part they show the machine is the **training** set; the hidden part is the **test** set. Later they quiz the machine with the leaves it never saw — that's the only honest way to know if it truly learned, or just memorized.

Size	Edges	Color	Label
large	rounded	green	Oak
small	pointy	red	Maple
small	rounded	green	Oak
large	pointy	orange	Maple
large	rounded	brown	Oak

← features (the clues) → the answer

A dataset is just a table: each row is one example, the feature columns describe it, and the last column holds the correct label.

Try It Yourself: Build Your Own Dataset

You don't need a computer to make a real dataset. You need a table and ten things.

1. Pick two kinds of something you can collect ten of: *coins and buttons, oak and maple leaves, spoons and forks* — five of each, ten total.
2. Draw a table. Make a row for each object, and three feature columns at the top, like **size**, **color**, and **shape**.
3. Fill in the features for every object — honestly! Then add one last column called **Label** and write what each thing actually is.
4. Now the test of variety: did you collect *only* big coins, or a mix? Only shiny ones? If a friend handed you a tiny dull coin, would your table still describe it?

That's it — you just built a labeled dataset by hand. Feed a table like this to a machine and it can start learning the difference. The wider your variety, the smarter it gets.

Watch Out

A dataset that's too small or too one-sided teaches the *wrong* lesson — and you won't notice until it's too late. Mango's machine looked fine on big sunny leaves, then fell apart the moment it met a small one in the shade. If you only train on one kind of example, the machine quietly decides that kind is the *only* kind. Always ask: “What does my data **not** include?” The leaves you forgot to collect are exactly the ones the machine will get wrong.

Word Bank

Data

The examples we feed a machine so it can learn. To a computer, data is rows of facts and answers.

Feature

One describable fact about an example — a leaf's size, color, or edge shape. Features are the clues the machine learns from.

Label

The correct answer attached to an example — “oak” or “maple.” The machine trusts every label, so they must be right.

Dataset

A whole collection of labeled examples, usually arranged as a table: one row per example, one column per feature, plus the label.

Bias

When lopsided data makes a machine unfair or blind to things it rarely saw. We'll meet this troublemaker again in its own class.

Big Idea

A machine never learns what you *want* it to learn — it learns whatever your data shows it. So feed it carefully.

Before You Turn the Page...

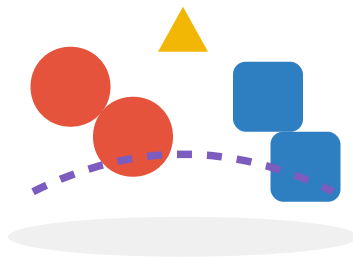
- **Data** is the food of AI: examples described by **features**, each with a correct **label**.
- Good data is **plentiful, varied, and correctly labeled** — quantity *and* quality.
- **Garbage in, garbage out**: one-sided data makes a blind, biased machine.
- Hide a **test** set the machine never sees, so you can check it learned for real.

Part Two begins! Now that the machine has good leaves to learn from, how does it actually decide? It plays a question game called a decision tree. →

PART TWO

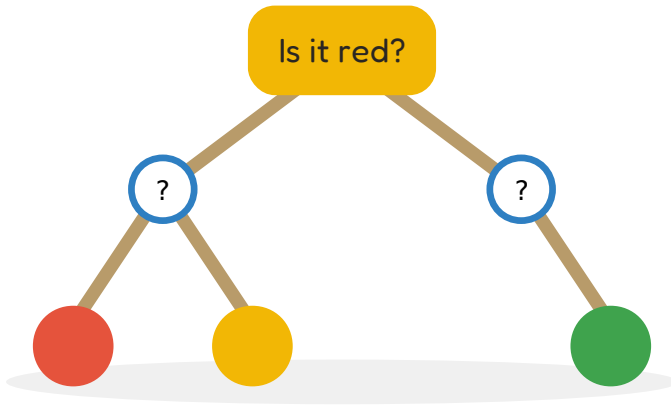
Sorting, Grouping & Guessing

Long before giant AI brains existed, scientists invented clever, simple recipes for letting machines make decisions. These are the classic algorithms – and they're still everywhere today. Get ready to play question games, find your flock, draw lines into the future, and learn why memorizing is not the same as understanding.



Decision Trees

The Question Game — how a machine makes a decision by asking the smartest question first.



Ziggy the zebra had a problem, and it was a fruity one. Professor Hoot had dumped an enormous basket on the classroom floor — apples, bananas, green pears, cherries, lemons, a single lonely blueberry — and asked the class to sort every piece into the right crate. There were three crates: *red things*, *yellow things*, and *green things*.

“Easy!” said Bolt the rabbit, and he began flinging fruit at top speed. A banana sailed into the red crate. A cherry bounced into the green one. Within thirty seconds he had made a magnificent, colorful mess.

Professor Hoot: “Bolt, you sorted fast. But did you sort well?”

Ziggy did something different. She didn't touch a single fruit yet. Instead, she stood very still and thought about the **best question to ask first**.

Ziggy: *"If I ask, 'Is it a cherry?', that only helps with the cherries. But if I ask, 'Is it red?' — that splits the whole basket almost in half in one go!"*

And that was the secret. Ziggy held up each fruit and asked one yes-or-no question at a time. *Is it red?* If yes, it went toward the red crate. If no, she asked a second question: *Is it yellow?* Yes meant the yellow crate; no meant green. Two small questions, and every single fruit landed in exactly the right place.

Mango: *"You built a little tree of questions in your head!"*

Professor Hoot: *"She certainly did. And that, class, is one of the oldest and most useful tricks in all of artificial intelligence. It's called a decision tree."*

Why the first question matters most

Here's the part Ziggy understood and Bolt didn't: not all questions are equally good. A great question is one that **splits a messy pile into neater piles**. "Is it red?" is a fantastic first question because it instantly separates apples and cherries from everything else. "Is it a blueberry?" is a terrible first question, because no matter the answer, you've barely sorted anything — there was only one blueberry!

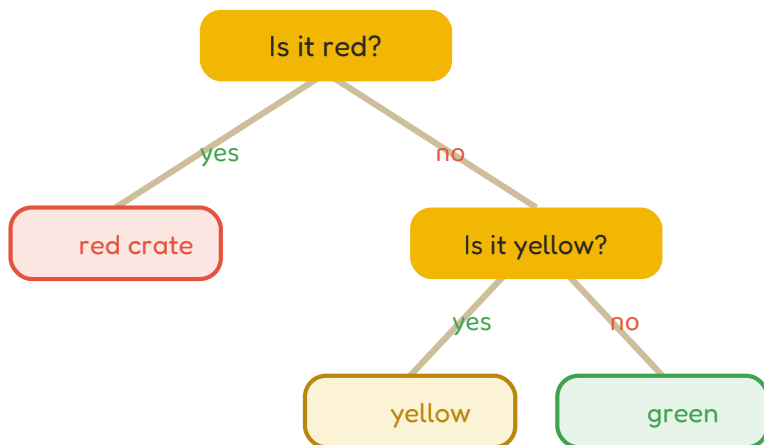
A decision tree always tries to ask the *most powerful question first*, then the next most powerful, and so on, until every pile is tidy.

How It Really Works

A real decision tree learns from **examples**. Imagine you give a computer a list of fruits, each one described by its *features* — color, size, whether it's bumpy — along with the right answer (its name).

The computer tests every possible question and measures how much each one **tidies up the mess**. Scientists have a name for the mess: they call it *impurity*. A pile with apples *and* bananas *and* pears all jumbled together has high impurity. A pile that's all apples has zero impurity — it's perfectly pure.

The tree picks the question that **reduces impurity the most**, splits the data on it, and then repeats the whole process on each new pile. It keeps going until each pile is pure (or close enough). That's it — that's the whole algorithm. No magic, just smart questions in a smart order.



Two well-chosen questions sort the whole basket. That's a decision tree.

The wonderful thing about a decision tree is that you can *read* it. If the machine sorts a fruit into the green crate, you can ask *why?* and it will tell you: “Because it was not red, and it was not yellow.” That makes decision trees one of the most **honest** kinds of AI — they never hide their reasoning. Doctors, banks, and scientists love them for exactly that reason.

Try It Yourself: Guess the Animal

Grab a friend and play the classic **20 Questions** game — but pay attention to your strategy. Your friend thinks of an animal. You may only ask yes-or-no questions.

1. Your first question should split the animal kingdom roughly in half. Good first questions: *“Does it live in water?”* or *“Is it bigger than a dog?”* Bad first question: *“Is it a hamster?”*
2. After each answer, pick the next question that splits what's left in half.
3. Count your questions. Can you guess any animal in 7 questions or fewer?

Congratulations — you just *were* a decision tree. Every good question you asked reduced the impurity of the “pile” of possible animals.

Watch Out

A decision tree can get *too* clever. If you let it keep asking questions forever, it will grow a giant tree with one tiny branch for every single example — basically memorizing the answers instead of learning the pattern. That's called **overfitting**, and it's such an important trap that it gets its own class later (Class 8). The fix is to stop the tree while it's still simple.

Word Bank

Decision tree

A model that makes a decision by asking a series of yes-or-no questions, branching like a tree until it reaches an answer.

Feature

One describable fact about a thing — a fruit's color, size, or bumpiness. Features are the clues a model gets to use.

Impurity

How mixed-up a pile is. The tree's whole job is to ask questions that lower impurity until each pile is neat.

Leaf

📌 The end of a branch, where the tree stops asking and gives its final answer.

Big Idea

A smart decision isn't about asking *many* questions — it's about asking the *right* question first.

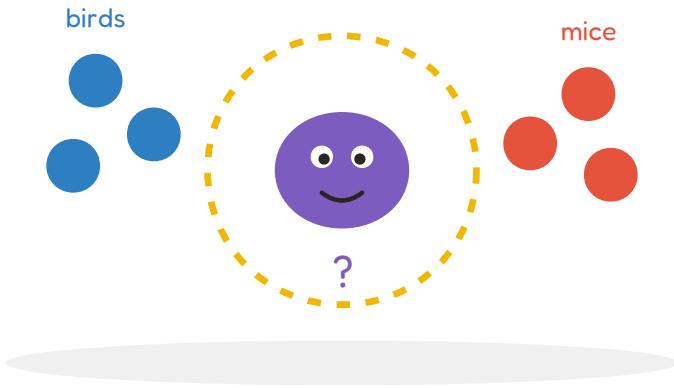
Before You Turn the Page...

- A **decision tree** sorts things by asking yes-or-no questions in a clever order.
- The best question is the one that **tidies the mess** the most.
- Trees are **easy to read** — they can always explain their reasoning.
- Let a tree grow too big and it starts **memorizing** instead of learning.

Next class: when there's no obvious question to ask, how does a machine guess? It looks at its neighbors. →

Nearest Neighbors

Birds of a feather — how a machine guesses what something is by looking at who it's most like.



Something was scratching at the Academy door. When Professor Hoot swung it open, in waddled a creature that no one — not one single animal in the whole classroom — had ever seen before. It was round and soft, a little bigger than a teacup. It had a tiny beak *and* whiskers. It made a sound halfway between a chirp and a squeak.

The room went quiet.

Mango: “What is it?”

Professor Hoot: *"That, Mango, is exactly the question. There is no sign on its back. It came with no instructions. So — how shall we figure out what it is?"*

Pip the mouse, who always noticed the small things, crept closer. She didn't try to make a rule. She just looked, very carefully, at the new creature, and then at everyone else in the room.

Pip: *"It's small, like me. But it has a beak, like Coco. Maybe... maybe we shouldn't guess from nothing. Maybe we should find out who it's most like."*

Coco the parrot: *"Birds of a feather flock together! Birds of a feather flock together!"*

Professor Hoot: *"Coco, you clever bird, you've just named the whole lesson."*

Let the neighbors vote

Here is what the class did. They picked a few simple things they could measure about any animal — its **size**, the **sound** it made, and what it liked to **eat**. Then they lined up the new creature next to the animals it resembled most.

By size, it sat near Pip and the other small ones. By sound, it landed somewhere between Coco's chirps and a mouse's squeak. By diet, it nibbled seeds — exactly like the birds. The class gathered the **five animals it was most similar to** overall and simply counted them up.

Pip: *"Of its five closest matches... three are birds, two are mice. Birds win the vote. I think our mystery friend is a kind of bird."*

Professor Hoot: “No rules. No long list of questions. You just found its nearest neighbors and let them vote. Class, you've discovered one of the friendliest algorithms in all of AI.”

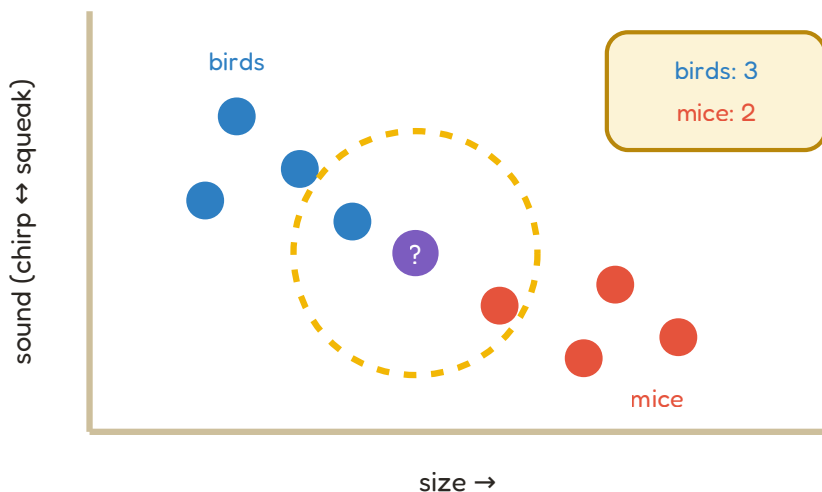
How It Really Works

This is the **k-Nearest Neighbors** algorithm, or *k-NN* for short. It works like a map. Every example you already know gets plotted as a dot, using its *features* as **coordinates** — size along one axis, sound along another. Similar animals end up sitting close together, so the birds form one little crowd and the mice form another.

When a mystery dot arrives, you measure the **distance** from it to every known dot — just how far apart they are on the map. You find the **k** closest dots (its nearest neighbors), look at their labels, and take a **majority vote**. Whatever most of the neighbors are, that's your guess.

That little letter **k** is the number of neighbors you ask. With $k = 1$ you trust the single closest dot — fast, but jumpy. With $k = 5$ you ask five and go with the crowd, which is steadier. People usually pick an **odd** number for k so the vote can't end in a tie.

Here's the surprising part: *k-NN* doesn't really *train*. There's no studying, no model being built. It simply **remembers every example** and, when a new one shows up, compares it to all of them on the spot. It's learning by remembering.



$k = 5$: of the five nearest dots inside the circle, three are birds and two are mice. Birds win the vote — so the “?” is labeled a bird.

Notice how lazy this algorithm is, in the best possible way. The decision tree from last class worked hard up front, building its questions. k -NN does almost nothing until you ask it something — then it springs to life, measures every distance, and votes. It keeps the whole crowd of examples in its pocket and pulls them out only when it's needed.

Try It Yourself: The Two-Feature Map

You'll need a big sheet of paper, a ruler, and a handful of classmates or family members.

1. Draw two lines for axes. Label the bottom one *height* and the side one *shoe size*.
2. For each person you know, put a dot where their height and shoe size meet. Color kids' dots blue and grown-ups' dots green.
3. Now think of a new person and plot their dot — but **don't** write the color yet.
4. Find the **three closest dots** ($k = 3$) using your ruler. Are most of them blue or green? That's your prediction: kid or grown-up.

Check if you were right. You just ran k-NN by hand, ruler and all.

Watch Out

Choosing k is a balancing act. Pick k **too small** — say $k = 1$ — and a single weird neighbor can fool you. One stray bird that wandered into the mouse crowd, and your guess goes wrong. Pick k **too big** and you drown out the local pattern: if you ask *everyone* in the room, the most common animal always wins, even when the answer is sitting right next door.

There's one more trap. Distance is fussy about numbers. If one feature is measured in huge values (say weight in *grams*: 5, 200, 4000) and another in small ones (number of legs: 2 or 4), the big-number feature bullies the measurement and the small one barely counts. The fix is **scaling** — shrinking every feature to a similar range first, so each one gets a fair vote.

Word Bank

Nearest Neighbors

The known examples that sit closest to a new one on the feature map. To classify the new one, you look at them.

Distance

How far apart two examples are when you plot them by their features. Smaller distance means more similar.

k

The number of neighbors you ask. An odd k (like 3 or 5) keeps the vote from ending in a tie.

Majority Vote

Letting the neighbors decide: whichever label most of them have becomes the prediction.

Classification

Putting a new thing into the right group — bird or mouse, kid or grown-up.

Big Idea

To guess what something is, you don't always need a rule — sometimes you just need to find who it's most like, and let the neighbors vote.

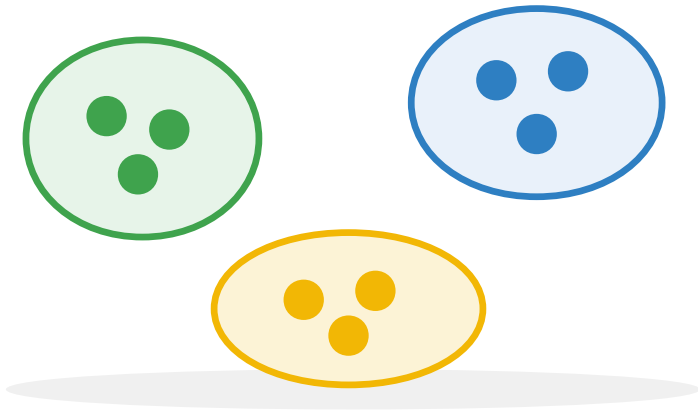
Before You Turn the Page...

- **k-Nearest Neighbors** classifies a new thing by finding the known examples most similar to it.
- Plot examples by their **features**, measure **distance**, and take a **majority vote** of the k closest.
- The letter **k** matters: too small is jumpy, too big drowns out the local pattern — and odd numbers avoid ties.
- k-NN never really trains — it just **remembers everything** and compares when asked.

Next class: what if nobody tells you the labels at all? The machine learns to find its own flocks. → Clustering: Finding Your Flock

Clustering

Finding Your Flock – how a machine discovers hidden groups when nobody ever told it what the groups are.



The bell rang for recess, and the whole Academy spilled out onto the big meadow at once. No teacher blew a whistle. No one read out a list of teams. No one said, “*You there – you’re on the climbers, and you over there, you’re a swimmer.*” The animals were simply set loose into the grass and told to play.

Ziggy the zebra found a quiet spot, sat down, and watched. She loved watching. And after only a minute, she noticed something that made her ears twitch.

Ziggy: “*Look! Nobody told them where to go... but they’re sorting themselves.*”

It was true. Over by the long stretch of flat grass, the fastest animals had gathered — they were racing each other end to end, a blur of legs. Up in the old oak, the climbers had found one another and were swinging branch to branch. And down by the pond, a splashing, happy huddle had formed: the swimmers. Three clear groups, and not a single grown-up had drawn the lines.

Bolt: *“Of course I’m with the runners!” he panted, skidding to a stop. “Where else would a rabbit go? Nobody had to tell me. I just... belong here.”*

Professor Hoot: *“And that, Bolt, is the whole lesson — though you ran right past it. In every class so far, I handed you the answers. ‘This is an apple, that is a banana.’ ‘This photo is a cat, that one is a dog.’ The labels came first.”*

Professor Hoot: *“But out here at recess, there are no labels at all. Nobody named a single group. And yet the groups appeared anyway — born out of how similar the animals are to one another. That, class, is called clustering.”*

The first lesson with no answer key

This was a brand-new kind of idea, and Ziggy felt it click into place. Up until now, every machine in the book had been shown the right answers and asked to copy the pattern. Teachers call that **supervised learning** — there's a supervisor handing over labels.

Clustering is different. There is no answer key. The machine is handed a big pile of things with *no* names, and its only job is to notice which ones naturally belong together. Because no supervisor is telling it the answers, this is called **unsupervised learning** — and it's one of the



most surprising tricks in all of AI. The machine finds the groups *by itself*.

How It Really Works

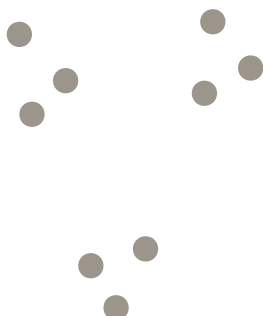
The most famous clustering recipe is called **k-means**, and it is shockingly simple. First a human picks one number: **k**, the number of groups to look for. Say we choose $k = 3$, because we suspect there are three kinds of recess animal.

Now the machine does four steps, over and over:

1. **Drop k center points.** It scatters 3 markers — called *centroids* — randomly onto the field of data.
2. **Assign each animal to its nearest center.** Every animal joins whichever centroid is closest to it. Now there are three rough groups.
3. **Move each center to the middle of its group.** Each centroid slides to the exact average position of all the animals that picked it.
4. **Repeat steps 2 and 3.** Animals re-pick their nearest center, the centers move again...

...until nothing changes anymore. When the groups stop shuffling, the machine is done — it has discovered the clusters on its own. But here's the honest part: the machine found *where* the groups are. It has **no idea** what they mean. A human still has to look and say, “Ah — this group is the swimmers.” And a human had to choose that there were 3 groups in the first place.

Before: just dots



After: 3 clusters



Same dots, twice. The machine finds the groups and parks a centroid () at the heart of each. Only a human can name them.

Ziggy grabbed a stick and drew it in the dirt for Bolt: a swarm of plain gray dots on the left, and the very same dots on the right – now colored into three neat clumps, each with a little star at its heart.

Ziggy: *“The star is the centroid – the middle of the group. The machine keeps nudging each star toward the middle of whoever’s nearest, until the stars stop moving. That’s when it knows the flocks have settled.”*

Mango: *“But how does it know to call you ‘swimmers’?”*

Ziggy: *“It doesn’t! That’s the funny part. The machine just says, ‘Here’s Group 1, Group 2, and Group 3.’ We walk over, take a look, and say, ‘Oh – Group 3 is soaking wet, let’s call them the swimmers.’ The machine finds the flock. The human gives it a name.”*

Try It Yourself: The Mystery Bag

Empty a junk drawer or a toy box onto the floor — buttons, LEGO bricks, odd socks, coins, bottle caps, whatever's in there. Make it a real mess. Then, **without anyone telling you the categories**, sort the pile into groups that feel like they belong together.

1. Just start grouping. Don't plan it. Trust your eyes.
2. When you're done, ask yourself two questions: *What did I group by — color? size? shape?* That feature you used is exactly what k-means measures to decide who's “near” whom.
3. And: *how many groups did I end up with — 3? 5?* Nobody handed you that number. You picked your own **k**.

Now try it again, but group by a *different* feature. The same pile splits into completely different clusters! Change the feature, change the flocks. You just ran k-means with your own two hands.

Watch Out

Clustering has a sneaky danger: it **always** finds groups — even when there aren't any. Toss a handful of totally random, meaningless dots at k-means and ask for 4 groups, and it will cheerfully hand you 4 groups. They just won't *mean* anything.

So **choosing k matters enormously**. Ask for too few groups and you'll squash swimmers and climbers into one mushy blob. Ask for too many and you'll split the runners into a dozen pointless little slivers. And remember: the machine never names the clusters — until a human reads them, “Group 2” means nothing at all.

Word Bank

Clustering

Sorting a pile of things into groups based only on how similar they are — with nobody saying what the groups should be.

Unsupervised learning

Learning from data that has *no* labels and no answer key. The machine has to find the structure all by itself.

Centroid

The center point of a cluster — the average position of everything in the group. k-means slides these around until they stop moving.

k

The number of clusters to look for. A human chooses k; the machine then finds that many groups.

Big Idea

Sometimes nobody tells you the groups — and a smart machine can still find the flocks hiding inside the crowd, just by noticing who belongs near whom.

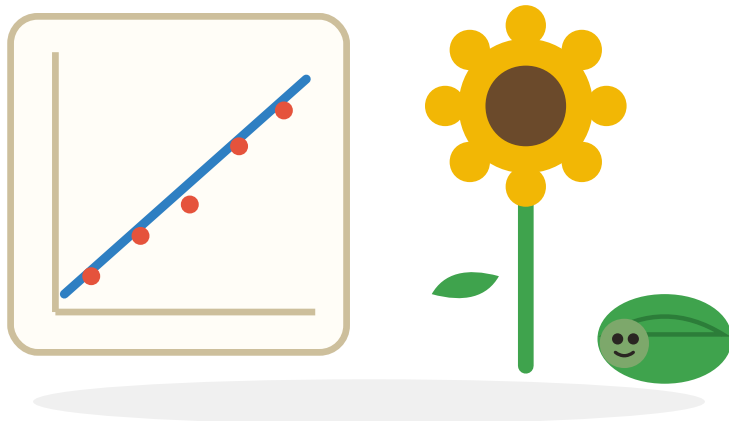
Before You Turn the Page...

- **Clustering** finds hidden groups when **no labels** were ever given.
- That's **unsupervised learning** — no answer key, just structure waiting to be found.
- **k-means** drops k centers, assigns each point to its nearest one, then moves the centers to the middle — over and over until they settle.
- The machine finds the groups; a **human** still picks k and decides what each cluster *means*.

Next class: instead of grouping the past, we draw a line straight through it — and follow it into tomorrow. →

Linear Regression

The Line Into Tomorrow – how a machine predicts a number by drawing the best straight line through your data.



The Academy was having a contest, and it was a slow, sunny, patient kind of contest – exactly the sort Tess the tortoise loved. Every student had planted a sunflower seed in a paper cup, and the rule was simple: grow the tallest sunflower by the end of summer. But Professor Hoot added one twist. Each Friday, everyone had to **measure** their plant and write down how tall it was.

Bolt the rabbit measured his once, decided it was “basically a tree already,” and never measured again. Tess did something else. Week

after week, she pressed her ruler against the stem and wrote a careful little number in her notebook.

Pip: *“Two centimeters in Week 1... five in Week 2... eight in Week 3... Tess, your numbers are so neat.”*

By Week 5, Tess had a column of numbers. She drew two lines on a fresh page — one going up the side for *height*, one going across the bottom for *week* — and placed a dot for each measurement. Then she leaned back and blinked.

Tess: *“Look. The dots... they almost make a straight line.”*

They did. The dots climbed upward in a tidy, slanted row, like steps on a gentle staircase. Not perfectly straight — one dot sat a hair high, another a hair low — but if you squinted, a single straight line could slide right through the middle of all of them.

Mango: *“So what? It's a nice line. Can it do anything?”*

Tess smiled her slow tortoise smile. She laid her ruler along the dots, drew the line, and kept drawing it — past her last dot, out into the empty future part of the chart, all the way to Week 8.

Tess: *“If the dots keep following this line, then in Week 8 my sunflower should be about... twenty centimeters tall. That's my prediction.”*

Predicting a number, not a category

Professor Hoot ruffled his feathers, delighted. “Class, remember when Ziggy sorted fruit into crates? She put each thing into a *category* — red, yellow, green. That's called **classification**. But Tess isn't sorting anything. She's guessing a *number* — how many centimeters tall. Guessing a number from a pattern has its own name. It's called **regression**.”

Three weeks later, the class gathered around Tess's cup with a ruler, holding their breath. Week 8. The sunflower stood tall and proud. Pip measured carefully: **twenty-one centimeters**. Tess had guessed twenty. Close — wonderfully, impressively close — using nothing but a notebook, a ruler, and a straight line.

How It Really Works

Tess just did **linear regression** — one of the most useful tools in all of machine learning. Here's the honest version.

Each measurement becomes a **dot** on a chart. One thing you know goes across the bottom (the *input* — the week), and the thing you want to predict goes up the side (the *number* — the height). Five measurements, five dots.

Now you draw a straight line through them. But not just any line — you want the one that comes **closest to all the dots at once**. For each dot, measure the gap between the dot and the line; that gap is called the **error**. A computer tries the line that makes the total error as *small as possible*. That special line has a name: the **line of best fit**.

Once you have the line, predicting is easy. Pick a new input — say, Week 8 — find it along the bottom, go up to the line, and read off the height. That's your prediction.

One gentle bit of math: a straight line has a **slope** (how steep it is — how many centimeters of growth per week) and a starting point (where it begins on the side). Learning the line just means finding the best slope and starting point. That's the whole trick. Classification picks a *category*; regression predicts a *number*.



Each dot is one week's measurement. The blue line fits the dots as closely as it can; following it past the data (dashed) gives the predicted height in Week 8.

That's why this idea shows up everywhere. The same straight-line trick predicts how much a house will cost from its size, how long a trip will take from its distance, or how much a plant drinks from how hot the day is. Whenever you have dots that lean in a direction, a line can carry the pattern forward into tomorrow.

Try It Yourself: Draw the Line Into Tomorrow

Pick something real that changes a little each day and track it:

1. Choose your number — a sprouting bean's *height*, the coins in your *savings jar*, or the *temperature* at the same time each afternoon.
2. Measure it once a day for at least five days and write down each number.
3. Draw two lines on paper: *day* across the bottom, your *number* up the side. Place one dot per day.
4. Lay a ruler down and draw the **straight line that passes as close to all the dots as you can** — about as many dots above it as below.
5. Keep the line going one day past your last dot. Where it lands is your **prediction**. Write it down.
6. Tomorrow, measure for real. How close were you?

Congratulations — you just fit a line of best fit by eye, exactly like Tess.

Watch Out

A line will happily keep going *forever* — but the real world doesn't. Tess's line says her sunflower grows about 2 cm every week. Follow that line far enough and it promises a sunflower taller than the school, then taller than a mountain, then poking the Moon. Of course that's silly. Real sunflowers slow down and stop.

Stretching a line way past your data is called **extrapolation**, and the farther you go, the less you can trust it. Predicting next week? Pretty safe. Predicting next *year*? Don't bet your lunch on it. And one more thing: not every pattern is a straight line — some dots curve. If your dots don't lean in a tidy slant, a straight line is the wrong tool.

Word Bank

Regression

Predicting a *number* from a pattern (like height in cm) — as opposed to classification, which picks a category.

Line of best fit

The single straight line that passes as close as possible to all your dots at once.

Error

The gap between a dot and the line. The best line makes the total error as small as it can be.

Slope

How steep the line is — how much the number changes for each step along the bottom (here, centimeters of growth per week).

Prediction

The answer you read off the line at a new input you haven't measured yet.

Big Idea

If your dots lean one way, the best straight line through them can carry the pattern forward — and turn yesterday's measurements into tomorrow's guess.

Before You Turn the Page...

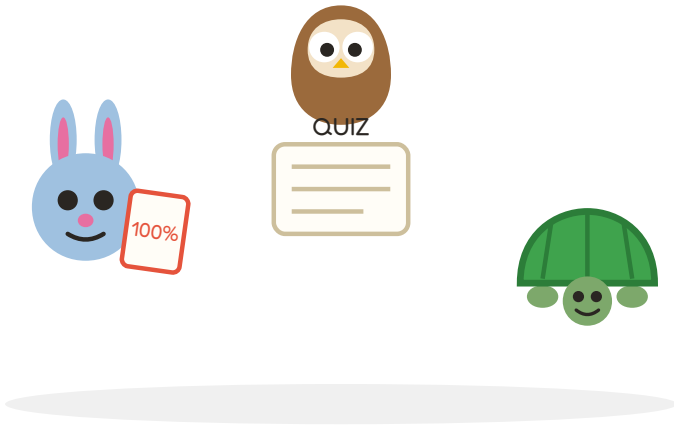
- **Regression** predicts a number; **classification** picks a category.
- Each example is a **dot**; the **line of best fit** comes as close to all of them as possible.
- “Closest” means making the total **error** — the gaps from dots to line — as small as it can be.
- You **predict** by reading the line at a new input — but stretch it too far and the prediction stops being trustworthy.

Next class: what if a model fits the dots too perfectly and just memorizes them? That's Overfitting: Memorizing Isn't Learning. →

CLASS EIGHT

Overfitting

Memorizing isn't learning — the difference between truly understanding an idea and just remembering the answers.



It was test week at the Academy, and the whole class was nervous — all except Bolt the rabbit, who was bouncing with confidence.

Professor Hoot had handed out a **practice quiz** the day before: five questions, with the answers printed neatly on the back. “Study these well,” the owl had said, “and tomorrow's real test will feel friendly.”

Bolt took one look at the practice answers and grinned. He didn't bother to understand *any* of them. Instead, he did what he was fastest at: he memorized. *Question one, answer “forty-two.” Question two,*

answer “the river.” By bedtime he could recite all five answers backward, forward, and in his sleep.

Bolt: *“I’ve got this! I know every single answer by heart.”*

Tess the tortoise studied differently — and slower, of course. She barely looked at the printed answers. Instead, she kept asking *why* each one was right. “If the answer is the river, what’s the *idea* behind it? When would the answer be the lake instead?” She wasn’t collecting answers. She was collecting the **reasons**.

Tess: *“I might be slow, Bolt, but I’d rather understand five questions than memorize five answers.”*

The next morning, Professor Hoot passed out the real test — and flipped the whole class upside down. The questions were **brand new**. Same ideas, completely different wording. Not one of them matched the practice quiz.

Bolt’s ears drooped. His memorized answers were useless — they fit the old questions perfectly and the new ones not at all. He’d scored a glorious 100% on the practice and now sat staring at a test he couldn’t touch. Tess, meanwhile, read each new question, recognized the underlying idea, and answered it calmly. She aced the test she had never seen.

Professor Hoot: *“Bolt, you didn’t learn the material. You memorized it. In my world, we have a name for that mistake. We call it overfitting.”*

Great on the old, useless on the new

Here is the trap, and it’s a sneaky one. Bolt looked like the best student in the room — right up until the questions changed. A model that has overfit is exactly like Bolt: it performs *brilliantly* on the examples it

studied, and then falls apart the moment it meets something new. The whole point of learning is to handle the new. Memorizing the old gets you nowhere.

How It Really Works

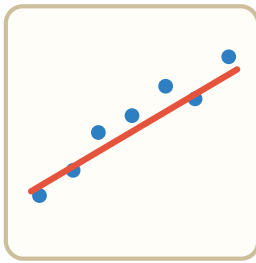
When we train a model, we feed it a pile of examples called the **training set**. The model's goal is not really to ace those examples — it's to learn the *pattern underneath* them so it can handle new examples it has never seen. That skill has a name: **generalization**.

Overfitting happens when a model hugs the training data *too* closely. It memorizes not just the real pattern but also every random bump, typo, and quirk — the *noise*. Like Bolt, it scores almost perfectly on what it has seen and then flops on anything new, because it learned the answers instead of the idea.

The opposite mistake is **underfitting**: a model so simple it misses the real pattern entirely. That's like a student who only learned “the answer is probably a number” — too vague to be right about anything. The goal is the **sweet spot in the middle**: complex enough to catch the true pattern, simple enough to ignore the noise.

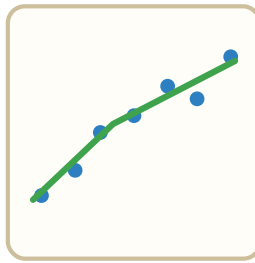
So how do we catch overfitting honestly, before it's too late? We hide some data. We split our examples into a **training set** (the model studies these) and a separate **test set** (the model never sees these while learning). Then we check the score on the test set — the questions it couldn't have memorized. That's the real test, exactly like Hoot's. And here's a surprising truth scientists rely on every day: a *simpler* model often generalizes better than a fancy, complicated one. Back in Class Four we warned that a decision tree grown too big starts memorizing instead of learning — that was overfitting, and now you know its full name.

Same dots, three different fits



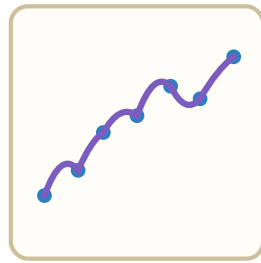
Underfit

too simple



Just right

learns the trend



Overfit

memorizes every dot

The straight line is too simple (underfit). The smooth green curve catches the real trend (just right). The wiggly purple line touches every single dot — including the noise — and will get lost on a new dot (overfit).

Look closely at that wiggly purple line. It passes through every dot *perfectly* — a flawless 100% on the dots it was given. But it's a fake. Drop a brand-new dot anywhere near the middle and the wiggly line will guess something silly, because it was busy hugging old dots instead of learning the gentle upward trend. The calm green curve misses a dot or two on purpose, and that's exactly why it will do better on everything that comes next.

Try It Yourself: The Multiplication Test

This experiment shows the difference between memorizing and understanding — using just a pencil and a friend.

1. **Be the "memorizing model."** Memorize these five facts and nothing else: $6 \times 7 = 42$, $8 \times 4 = 32$, $9 \times 3 = 27$, $7 \times 5 = 35$, $6 \times 6 = 36$. Don't think about *how* multiplication works — just chant the five answers until you know them cold.
2. **Now be the "understanding model."** On a different day, learn what multiplication actually *means*: 6×7 is six groups of seven, which you can count up or add.
3. **Take the real test.** Have your friend ask you *different* facts you never memorized — 8×6 , 7×9 , 4×5 . The memorizing version is stuck. The understanding version can work out every single one.

Which "model" generalized? The one that learned the idea, not the answers. That's the whole lesson of overfitting in one pencil-and-paper test.

Watch Out

If a model scores a **perfect 100% on its training data**, do not cheer — get suspicious. A perfect training score is usually a *red flag*, not a trophy. It often means the model memorized the examples (noise and all) instead of learning the pattern. The only honest scoreboard is a **test set** the model has never, ever seen. If it does well on fresh, unseen questions — *then* you can celebrate.

Word Bank

Overfitting

When a model fits its training examples too closely — memorizing even the random noise — so it does great on what it has seen and badly on anything new.

Underfitting

The opposite mistake: a model too simple to catch the real pattern, so it does poorly on everything, old and new alike.

Training set

The examples a model studies and learns from — its practice quiz.

Test set

A separate batch of examples the model never sees while learning, kept aside to check honestly whether it really understood — its real test.

Generalization

The goal of all learning: handling new, unseen examples correctly, not just the ones you studied.

Big Idea

Learning isn't remembering the answers — it's understanding the idea well enough to answer questions you've never seen before.

Before You Turn the Page...

- **Overfitting** means fitting the training data so closely you memorize its noise and fail on anything new.
- **Underfitting** is the opposite — too simple to catch the pattern at all. The goal is the sweet spot in between.
- We split data into a **training set** and a separate **test set** so we can check learning honestly.
- A perfect training score is a **red flag**; simpler models often generalize better.

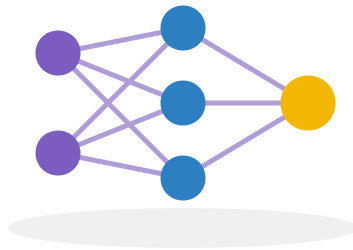
Part Three begins! Next class: what if a machine could learn the way a brain does, with thousands of tiny connected cells? Meet neural networks. →

PART THREE

Brains Made of Math

Now things get really exciting. We're going to build a brain – not out of cells, but out of tiny math switches called neurons. You'll see how stacking them into layers lets a machine learn to recognize a cat, read handwriting, and even remember what came before.

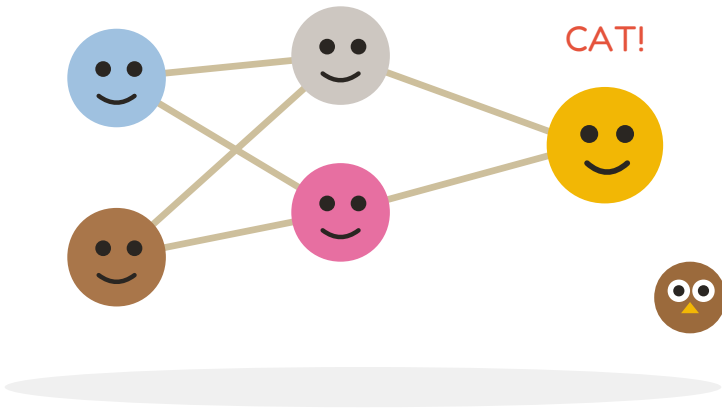
This is the part of AI that changed the world.



CLASS NINE

Neural Networks

How to build a brain out of thousands of tiny, simple switches — none of them clever, all of them working together.



Professor Hoot pushed all the desks against the wall and clapped his wings. “Today,” he announced, “we are not going to *read* about a brain. We are going to *become* one. Everybody up. Line up in rows.”

The class shuffled into three neat rows facing the chalkboard. Bolt was in the front. Pip and Mango were in the middle. Coco stood alone at the very back.

Professor Hoot: *"Each of you is now a tiny part of a brain. We call one of these tiny parts a neuron. And here is the most important rule of the whole day: your job is incredibly dumb."*

Mango: *"Dumb? I came here to get smarter!"*

Professor Hoot: *"Patience. Here is your one and only job. Listen to the numbers that get shouted at you. Add them up. If the total is big enough to be exciting – shout to the row behind you. If it's too small, stay quiet. That's it. That's everything you do."*

Bolt frowned. "That doesn't sound like thinking at all."

Professor Hoot: *"One of you, alone, can barely do anything. But watch what rows of you can do together."*

The Professor wrote a secret drawing on a card – a scribbly creature with whiskers and pointy ears. Then he fed numbers about the drawing to the front row: *"Pointy ears: a 9! Whiskers: an 8! A long flat beak: a 0!"*

Bolt heard "pointy ears: 9" and a couple of other numbers. He added them up. The total felt exciting. So Bolt shouted a big number to the middle row. Next to him, an animal who only heard small, boring numbers stayed perfectly quiet.

The middle row didn't see the drawing at all. They only heard how loudly the front row was shouting. Pip listened – lots of front-row animals were yelling "ears!" and "whiskers!" at once. *That* felt exciting. So Pip shouted to the back.

Finally Coco, all alone at the back, added up everything she heard from the middle row. The total was huge. She took a breath and yelled the answer the whole network had been building toward:

Coco: *"It's a CAT!"*

The Professor flipped the card around. It was a cat. The room erupted.

Nobody in the room knew it was a cat

Here's the spooky part, and the Professor made sure everyone caught it. Bolt never knew it was a cat — he only knew “pointy ears: 9.” Pip never knew either — she only knew “the front row is yelling a lot.” Not one single animal understood the whole picture.

Professor Hoot: *“And yet together you recognized a cat. Each of you did one tiny multiply-add-and-shout. Stacked into rows, those dumb little steps added up to something genuinely clever. That, class, is a neural network — a brain built out of math.”*

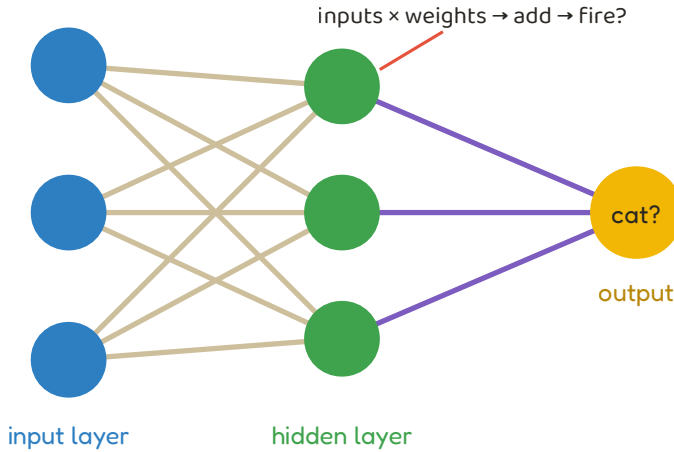
How It Really Works

A real **neuron** takes in several numbers — its *inputs*. But it doesn't treat them all the same. Each input is multiplied by a **weight**, a number that says *how much this input matters*. A big weight means “pay close attention to this one.” A weight near zero means “ignore it.” The neuron multiplies every input by its weight, then adds all those products into one total.

Then comes the decision. The total is passed through a tiny step called an **activation**, which decides how strongly the neuron should “fire.” A common one is simple: if the total is above a threshold, fire; if not, stay quiet — exactly like the animals shouting or staying silent.

One neuron is almost useless. The power comes from stacking them into **layers**. The *input layer* takes the raw facts (the pixels of a drawing). One or more *hidden layers* in the middle combine those facts into bigger ideas (“this looks like an ear,” “this looks like a whisker”). The *output layer* gives the final answer (“cat” or “not cat”). Signals flow **forward**, row by row, from input to output.

So where is the network's “knowledge”? It lives entirely in the **weights** — those millions of little numbers. Learning means *adjusting the weights* until the answers come out right. (How a network tunes its own weights is the whole story of the next class.) And **deep learning**? That fancy phrase just means a network with *many* hidden layers stacked deep.



Signals flow left to right. Every line is a weight — and all the network's knowledge lives in those numbers.

Mango raised a hand. “So if I want the network to recognize a dog instead of a cat... I just change the numbers on the lines?”

Professor Hoot: “Exactly right. Same animals, same rows, same dumb little jobs. You only change the weights — which signals get shouted loudly and which get hushed. Get the weights right and the very same network can spot a dog, read your handwriting, or steer a car.”

Try It Yourself: Be a Human Network

You need a handful of friends and a little open space. Line them up in two or three **rows**, all facing the same way.

1. Give each person in the **front row** a secret number on a card. Tell them the rule: *"If your number is bigger than 5, raise your hand."* That's their activation.
2. Each person in the **next row** can't see the cards — they can only *count the raised hands* in front of them. Their rule: *"If you can count 2 or more raised hands, you raise yours too."*
3. Add one more row with the rule *"Raise your hand if anyone in front of you did."* The **last person** raising a hand is the network's final decision.

Now change one front-row card and run it again. Watch a single tiny change **ripple** forward through the rows to flip the final answer. You just ran a feed-forward neural network made of people.

Watch Out

A neural network is *not* a real brain, and it is **not conscious**. It doesn't feel, want, or understand anything. Under all the excitement it is just a big stack of the same dull step over and over: multiply, add, and decide whether to fire. When Coco shouted “CAT!” she had no idea what a cat is — she only had a very large number.

And don't be fooled by any single neuron. One on its own can barely tell big numbers from small ones. **All** the cleverness comes from *many* of them, wired together in layers.

Word Bank

Neuron

One tiny unit in the network. It takes several numbers in, multiplies each by a weight, adds them up, and decides how strongly to fire.

Weight

A number on each connection that says how much an input matters. All of the network's knowledge is stored in its weights.

Layer

A row of neurons. Networks stack an input layer, one or more hidden layers, and an output layer; signals flow forward through them.

Activation

The little step that decides how strongly a neuron fires once its inputs are added up — like “shout if the total is exciting enough.”

Deep learning

Simply a neural network with many hidden layers stacked deep. More layers let it learn more complicated patterns.

Big Idea

Thousands of tiny, dumb switches — each just adding numbers and deciding whether to fire — can together do something genuinely smart.

Before You Turn the Page...

- A **neuron** does one simple job: weigh its inputs, add them up, and decide whether to fire.
- Neurons are stacked in **layers**, and signals flow forward from input to output.
- All the network's knowledge lives in its **weights** — the numbers on the connections.
- It's not a real brain and not conscious — just a huge pile of multiply-add-and-decide steps working together.

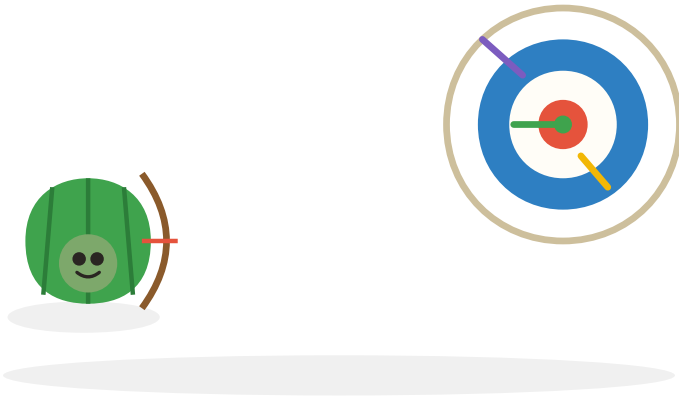
Next class: a fresh network's weights are pure guesswork. So how does it get good? It practices — millions of times. How a Network Learns:

Practice Makes Better. →

CLASS TEN

How a Network Learns

Practice makes better — getting closer by measuring your mistake, then nudging your aim a tiny bit at a time.



Last class, the forest built its very first neural network — a web of little decision-makers all wired together. But a fresh network is like a brand-new student: full of connections, and absolutely *terrible* at everything. Its **weights** — the dials that say how loudly each connection matters — start out as random numbers. Today the class had one job: teach the network to actually be *good*.

For this lesson, Professor Hoot set up an archery range in the meadow. Tess the tortoise volunteered first. She is slow, careful, and never rushes — which, it turns out, is exactly the right personality for learning.

Tess drew the bow and let an arrow fly. It thudded into the grass, a good arm's length to the *left* of the target.

Bolt: *“Ha! A miss! Let me try — I’ll just fire a hundred arrows really fast and one of them is bound to hit.”*

Professor Hoot: *“Patience, Bolt. Watch what Tess does next. She is not going to fire blindly. She is going to study her mistake.”*

Tess looked hard at where the arrow had landed. She asked herself two questions, and only two: **How far off was I?** and **Which direction?** Too far left, by a lot. So she nudged her aim a little to the right — not all the way, just a small, careful adjustment — and shot again.

This time the arrow landed left of the bullseye too, but *closer*. Less wrong. Tess nudged right again, by an even smaller amount. Closer still. Shot after patient shot, the misses shrank and shrank until — *thunk* — an arrow buried itself dead in the red center.

Mango: *“She didn't get lucky... she got less wrong every single time!”*

Professor Hoot: *“Precisely. And that loop — try, measure how wrong you are, adjust a little, repeat — is exactly how a neural network learns. Tess just showed us the whole secret.”*

Measuring the mistake

Here is the part most people miss: Tess couldn't improve until she could **measure** her mistake as a number. “A bit to the left” is a feeling. “Forty centimeters to the left” is something you can shrink. A network needs the same thing — a single number that says *how wrong* its guess was. The bigger the number, the worse the guess. The whole goal of learning is to make that number as small as it can possibly be.

How It Really Works

A neural network learns by repeating four steps, over and over:

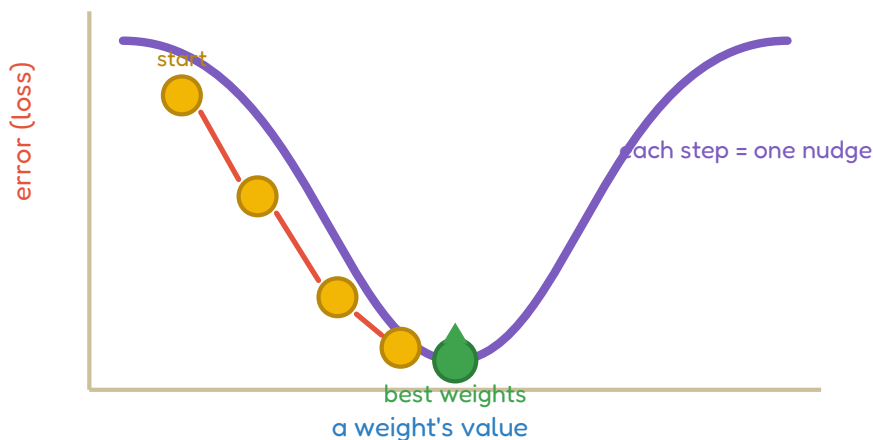
1. Guess. You feed in an example, and the network — using its current weights — produces an answer. At first the answer is basically random.

2. Measure the mistake. We compare the guess to the correct answer and boil the difference down to one number called the **loss** (also called the *error*). High loss means a bad guess. We want loss as close to zero as possible.

3. Figure out which way is “downhill.” For each weight, we ask: *if I nudged this dial up a hair, would the loss get bigger or smaller?* That answer — the direction that makes the loss *shrink* — is what mathematicians call the **gradient**. Following it is like standing on a foggy hill and feeling with your foot for which way slopes downward, then stepping that way. That's why the method is named **gradient descent**: descend the error.

4. Nudge a little, then repeat. We tweak every weight a small step in its downhill direction. How big a step? That's the **learning rate**. Then we do it all again with the next example. One full pass through all the examples is called an **epoch**, and a network usually needs *many* epochs to get good.

The clever trick that figures out the downhill direction for *every* weight at once — even the ones buried deep in the middle of the network — is called **backpropagation**. It starts at the final error and works *backward* through the layers, handing each weight its share of the blame. Then gradient descent nudges them all. That's the engine. Everything else is repetition.



Picture the error as a valley. The ball (the network's weights) feels which way is downhill and takes a small step. Step by step, it rolls to the bottom — the lowest error, the best weights.

Notice the steps in the picture get *smaller* near the bottom —

N just like Tess's nudges got tinier as her arrows closed in on the bullseye. That's no accident. When the slope is steep, the network is very wrong, so it takes a bigger step. As the valley flattens out near the best answer, the steps naturally shrink, and the network settles gently into place instead of crashing past it.

Tess: *"Slow and steady. I never tried to fix the whole mistake in one shot. I just made each shot a little less wrong than the last."*

Try It Yourself: Hotter / Colder

You already know this game — and it turns out you've been doing gradient descent your whole life. Grab a friend and a small hidden object (a coin, a toy).

1. Your friend hides the object somewhere in the room while you cover your eyes.
2. Now you walk around. Your friend's *only* job is to call out “**warmer!**” when you move closer and “**colder!**” when you move farther away. “Warmer/colder” is your *loss* — your measured error.
3. Use the feedback: keep stepping in whatever direction makes it warmer. That direction is your **gradient**. The size of your steps is your **learning rate**.

Count how many moves it takes to find the object. Then play again taking *giant leaps*, and again taking *tiny shuffles*. Which gets there fastest? You're about to discover why the learning rate matters so much.

Watch Out

The learning rate is a Goldilocks number — it has to be *just right*.

Set it **too big** and the network takes wild, giant leaps. It overshoots the bottom of the valley, flies up the far side, overshoots back, and bounces around forever without ever settling — like sprinting across the room and blowing right past the hidden coin every single time.

Set it **too small** and learning crawls. The network inches downhill so slowly it might take thousands of epochs to get anywhere — Tess's careful patience taken to a painful extreme.

And one more trap: a valley can have little dips that aren't the *deepest* spot. A network can roll into one of these and get stuck in a “pretty good but not best” place, thinking it's done. Researchers have clever tricks to nudge it back out.

Word Bank

Loss (Error)

A single number that measures how wrong the network's guess was. Big loss means a bad guess; learning means shrinking it toward zero.

Gradient descent

The method of finding which way is “downhill” for the error and taking a small step that way — over and over — until the loss is as low as it can get.

Learning rate

How big each nudge is. Too big and it overshoots and bounces; too small and it learns painfully slowly.

Epoch

One full pass through all the training examples. A network usually needs many epochs to get good.

Backpropagation

The trick that works backward from the error to figure out how each weight, even the hidden ones, should be nudged.

Big Idea

You don't fix a mistake all at once — you measure how wrong you are, step a little toward better, and repeat. Do that enough times and “terrible” quietly becomes “great.”

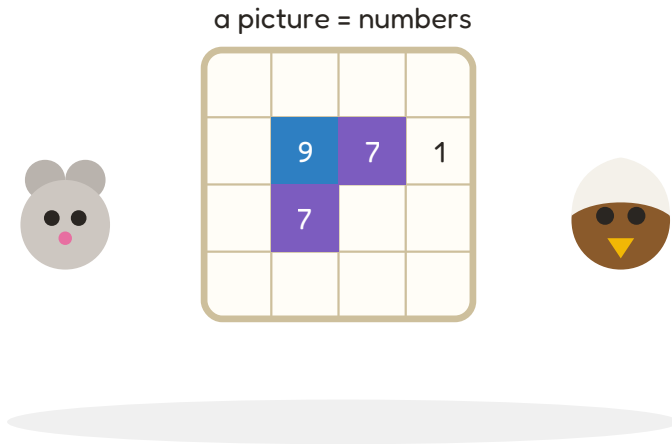
Before You Turn the Page...

- A network learns by a patient loop: **guess, measure the error, nudge the weights a little, repeat.**
- The **loss** turns “how wrong” into a single number we can shrink.
- **Gradient descent** rolls the error downhill; **backpropagation** figures out how to nudge every weight.
- The **learning rate** must be just right — too big overshoots, too small crawls.

Next class: now that our network can learn, let's give it eyes. Computer Vision: Teaching Machines to See. →

Computer Vision

How can a machine "see," when all it ever gets is a giant grid of numbers?



Professor Hoot held up a photograph of a fluffy gray cat. “Tell me,” he said, “what is this?” The whole class shouted at once: “A cat!” Easy. Pip the mouse, who notices things nobody else does, smiled — she’d already counted its whiskers.

“Good,” said the owl. “Now watch.” He tapped the photo, and on the big screen it began to *zoom*. Closer. Closer. The cat’s soft ear blurred into a fuzzy patch. The patch broke into blobs. The blobs broke into... **tiny colored squares**. Zoomed all the way in, the cat was gone. There was nothing left but a grid of little squares, each one a single flat color.

Professor Hoot: *"This, class, is what a computer actually sees. Not a cat. Not even a picture. Just a grid of dots — and every dot is only a number telling how bright it is."*

A new friend had flown in for this class: **Skye**, a sharp-eyed eagle who can spot a fish from the top of a tree. Skye squinted at the screen.

Skye: *"So a picture is secretly a spreadsheet? Every little dot is a number — a 0 for black, a 9 for bright white, and everything in between?"*

Professor Hoot: *"Exactly so. Each tiny square is called a pixel. A phone photo has millions of them. To the computer, the whole image is just a colossal grid of numbers. So here is our puzzle..."* He looked straight at Pip. *"If all the machine gets is numbers, how on earth does it ever say cat?"*

Build the big thing from small parts

Pip went very quiet, the way she does when she's thinking hard. "When I look at a cat," she said softly, "I don't see 'a cat' all at once. First I notice little things. A pointy edge here. A curved line there. Then I notice those edges make a triangle — an *ear*. Two round shapes — *eyes*. A row of straight lines — *whiskers*. And only *then*, after I've spotted all the little parts, my brain says: cat!"

Professor Hoot: *"Pip, you have just described how a machine learns to see. It does not try to recognize a whole cat in one giant leap. It starts with the simplest parts and builds up."*

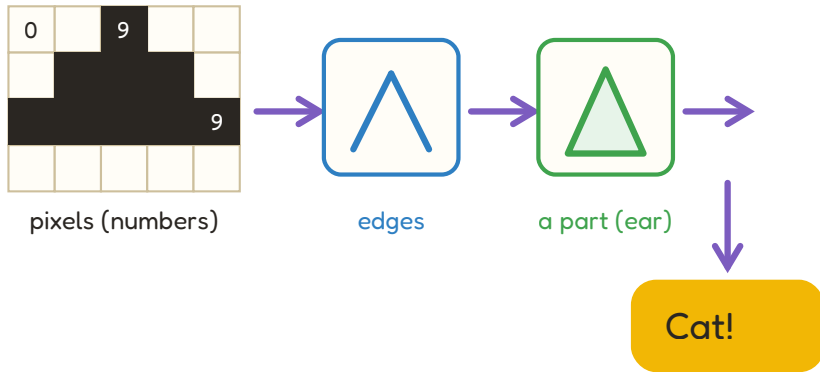
Edges first. Then edges joined into small shapes — a corner, a curve. Then shapes joined into bigger parts — an ear, an eye, a nose. Then all the parts together — a face, a body, a whole cat. Small things stacked into bigger things, layer by layer. Skye nodded slowly. "That's how I hunt, too. First I spot a flash of movement. Then a fin. Then — fish!"

How It Really Works

To a computer, an image is a **grid of pixels**, and each pixel is just a number for its brightness (a color pixel is three numbers — red, green, blue). That's the whole input: rows and rows of numbers.

To find patterns in those numbers, the machine uses a special kind of neural network — the same idea from Classes 9 and 10, but built for grids. It's called a **convolutional neural network**, or CNN. Its trick is a little **filter** (also called a *feature detector*): a tiny window, maybe 3×3 pixels, that **slides across the whole image**, one patch at a time, lighting up wherever it finds the pattern it's looking for. One filter might fire on vertical edges; another on a patch of green; another on a corner.

Here's the beautiful part. The **early layers** of filters find *simple* features — edges, corners, blobs of color. The next layer combines those into **shapes** — a circle, a curve, a stripe. Deeper layers combine *those* into **parts** — an eye, an ear, a wheel. And the final layers combine the parts into whole **objects** — a cat, a car, a face. Nobody tells the network what an edge or an ear is; it *learns* the useful filters from thousands of example pictures. Stacking many layers is exactly why these *deep* networks are so astonishingly good at vision.



simple features → bigger features → whole object

A computer sees only numbers. Layer by layer, it builds edges into shapes, shapes into parts, and parts into a whole cat.

Skye: “So the filters are like my eyes' first instinct — they hunt for the small clues — and the deeper layers are my brain putting the clues together.”

Professor Hoot: “A fine way to picture it. And notice why Pip is so good at this. She is patient with small parts. A machine that sees must be patient with small parts too — because the small parts are all it has.”

Try It Yourself: Become a Pixel Artist

Turn a picture into data the way a computer does it. You'll need graph paper (or draw a grid of squares — 10×10 is great) and a pencil.

1. Draw something simple inside the grid by **filling whole squares**: a heart, a smiley, the letter of your name. Only fill a square if your shape covers most of it.
2. Now write a **number in every square**: **0** for an empty (white) square, **9** for a fully filled (black) square, and something in between (like 4 or 5) for squares you only shaded a little.
3. You just turned a picture into a *grid of numbers* — the exact thing a computer "sees."
4. Trade grids with a friend. Read *only* the numbers, not the drawing, and try to color in each square by its number. Can you decode their picture?

If your friend can rebuild your drawing from numbers alone, you've proved that a picture really is just data.

Watch Out

Because a vision AI only ever sees numbers, it can be **fooled** in ways that would never trick you. Researchers have changed a few pixels in a photo — so few that a human sees no difference at all — and watched a confident AI suddenly call a panda a gibbon. These sneaky tricks are called *adversarial examples*.

And remember: a CNN only recognizes things **similar to what it was trained on**. Show it a cat in a silly costume, a cat upside down, or a kind of cat it never studied, and it may be completely stumped. It learned patterns from its examples — it doesn't truly "understand" cats the way you do.

Word Bank

Pixel

One tiny dot of a picture. To a computer it's just a number (or three) saying how bright and what color that dot is.

Computer vision

The field of teaching machines to make sense of images and video — to find and recognize what's in a picture made of pixels.

Filter (feature detector)

A small window that slides across the image hunting for one simple pattern, like an edge or a corner, lighting up wherever it finds it.

Convolutional neural network (CNN)

A neural network built for grids of pixels. It slides many filters over an image and stacks them in layers to recognize objects.

Feature

A useful clue the network finds — from simple ones (edges) in early layers to complex ones (an eye, a whole face) in deeper layers.

Big Idea

A machine doesn't see a cat — it sees a grid of numbers, and learns to build the cat out of edges, then shapes, then parts, one layer at a time.

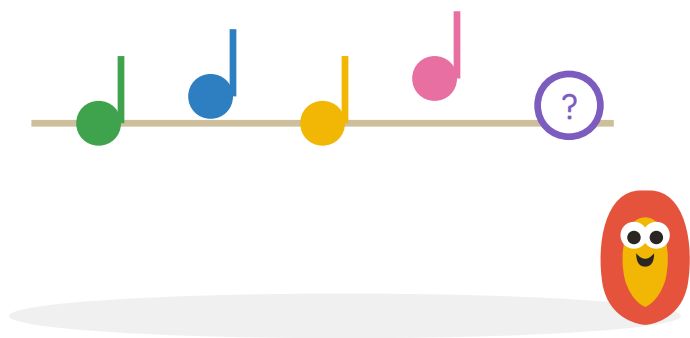
Before You Turn the Page...

- To a computer, every picture is just a **grid of pixels** — a giant grid of numbers.
- A **CNN** slides little **filters** across the image to find simple patterns like edges.
- **Early layers** find simple features; **deeper layers** combine them into shapes, parts, and whole objects.
- Vision AI can be **fooled** by a few changed pixels, and only knows things like what it was trained on.

Next class: pictures stand still, but words and music come in order. How does a machine remember what came before? Memory & Sequences. →

Memory & Sequences

Remembering what came before — how an AI predicts what comes next when the order matters.



Coco the parrot could not sit still when there was music in the room. The moment Professor Hoot tapped out a tune on the old classroom piano, her head began to bob and her feathers fluffed up with joy. Coco *loved* songs — and, more than that, she remembered them. Hum half a melody anywhere in the forest and Coco would finish it for you.

So Professor Hoot decided to set her a little test. He played four soft notes — *low, high, low, higher* — and then, right in the middle of the fifth, he stopped. His wing froze above the keys.

Professor Hoot: “Coco. What’s the next note?”

Coco didn't even hesitate. She sang it — a bright, rising note — and it was exactly right. Tess the tortoise blinked, slowly, the way Tess does.

Tess: *“How did you know? He never played that note.”*

Coco: *“I didn't know it. I guessed it — from the notes that came before. The song was climbing. So the next note climbs too.”*

Professor Hoot ruffled with delight. He pressed the same five keys again, but this time in a jumbled order. It sounded like a sneeze. Nobody's head bobbed.

Professor Hoot: *“Same notes, Coco. Why isn't it the same song?”*

Coco: *“Because the order changed! A song isn't just a bag of notes. It's notes in a line. Mix up the line and you've got a different song — or no song at all.”*

The world is full of lines

Here is the big secret Coco understood in her feathery little heart: lots of things in the world come as **sequences** — ordered lines where the position of each item matters. A song is a sequence of notes. A sentence is a sequence of words. The weather is a sequence of days. Your morning is a sequence of steps (socks usually go on *before* shoes).

And to guess what comes *next* in any of these, you have to remember what came *before*. You need **memory**. Coco's gift for holding a sequence in her head is the very same trick that taught machines to finish our sentences and write little tunes of their own.

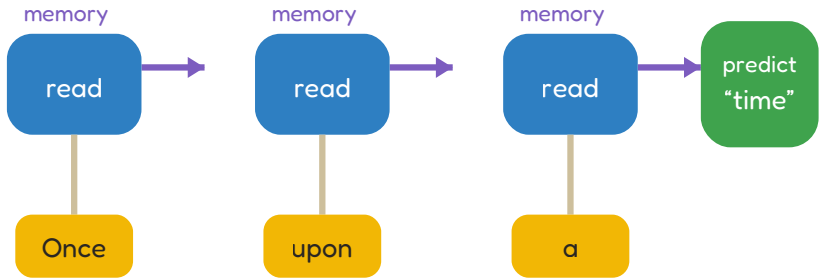
How It Really Works

A **sequence** is an ordered list where the *position* of each item matters: cat and act use the same letters, but they are not the same word. To predict the next item, a machine reads the sequence **one item at a time**, left to right.

The clever part is that, as it reads, it keeps a little pocket of **memory** — engineers call it the *state*. The state is a short summary of everything the machine has seen so far. At each step it does two things at once: it uses *state + the new item* to **predict** the next item, and it **updates** its state to fold the new item in. Then it loops. Read, remember, predict. Read, remember, predict.

Because it loops back on itself this way, this kind of model is called a **Recurrent Neural Network**, or RNN. (Remember the *neural network* from an earlier class? An RNN is one of those, wired so a piece of its output feeds back in as memory.) For years, RNNs were how computers handled language and music — reading a sentence word by word and guessing the next word, just like Coco guessing the next note.

But the memory is small, and it **fades**. Each new item nudges the old summary aside, so something from way back at the start of a long sequence can slip away before the machine reaches the end. Fixing that fading memory is exactly what the next big idea — *attention* — was invented to do.



It reads a word, passes its memory along, and the last box guesses what

The memory arrow (purple) carries a summary of every word so far. That's how the machine knows "Once upon a ..." should be followed by "time."

Notice what the chain is doing. The first box reads *Once* and remembers it. The second box reads *upon* but also gets the memory of *Once*. The third reads *a* with the memory of *Once upon* tucked inside. By the time we reach the end, the machine isn't staring at a single lonely word — it's holding the whole little story so far. That's why it can confidently sing out "time."

Try It Yourself: The Story Chain

This is a memory game you can play with one friend or a whole table of them.

1. Start a sentence with two or three words, out loud: “*Once upon a ...*” The next player must add **exactly one** word that fits.
2. Keep going around. Each new player has to remember the *whole* sentence in order before adding their word. No writing it down!
3. When someone forgets the order, or adds a word that doesn't fit what came before, the chain breaks and the round ends.

Try a number version too: **2, 4, 6, 8, ?** — what comes next? Or **red, orange, yellow, ?** To answer, you used your *memory* of the earlier items to spot the pattern. You just *were* a sequence model. And notice how much harder it gets to remember the start once the sentence runs very long...

Watch Out

Small memory **forgets the beginning**. Read this: “The **parrot** who lived in the tall green tree by the river that the woodpeckers loved ... *sang*.” By the end you may have lost track of who was doing the singing! Simple RNNs have the same trouble: with only a little state, each new word crowds out the older ones, so facts from long ago fade before they're needed. This real weakness is exactly why smarter methods came next.

Word Bank

Sequence

An ordered list of items where the *position* matters — a song, a sentence, a row of days. Shuffle the order and you change the meaning.

Memory (state)

A little pocket of information a model keeps as it reads, summarizing everything it has seen so far so it can use the past to guess the future.

Recurrent Neural Network (RNN)

A neural network that reads a sequence one item at a time and feeds part of its output back in as memory — looping so the past flows forward.

Prediction

📌 The model's best guess at the next item in the sequence, made from its memory plus the item it just read.

Big Idea

To guess what comes *next*, a machine has to remember what came *before* — and keep that memory in order.

Before You Turn the Page...

- A **sequence** is an ordered line of items, and the order changes the meaning.
- A sequence model reads one item at a time, keeping a small **memory** of what it has seen.
- It uses that memory to **predict** the next item — these loop-back models are called **RNNs**.
- Their memory **fades**, so long-ago things get forgotten — a problem waiting to be fixed.

Welcome to Part Four. Next class: when a machine remembers everything you said, it can finally talk back — **Language Models: How AI Learned to Talk.** →

PART FOUR

Machines That Talk & Decide

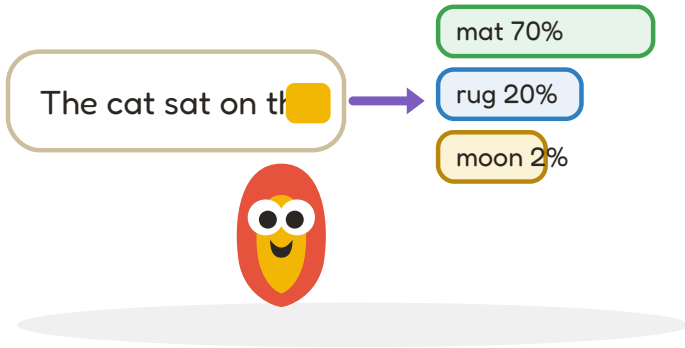
These are the AIs you've actually met — the chatbots that write, the apps that recommend, the game characters that find their way. In this part we'll crack open language models, learn the trick called attention, train a machine with treats, and watch an algorithm solve a maze. This is AI as you see it in the world right now.



CLASS THIRTEEN

Language Models

How a chatbot writes a sentence — by playing the world's biggest guess-the-next-word game.



Coco the parrot had picked up a brand-new habit, and it was driving everyone slightly bananas. She had started **finishing other people's sentences**.

Mango: "Hey Coco, can you pass me the —"

Coco: "— BANANA! Pass me the banana!"

Pip: "Once upon a —"

Coco: "— TIME! Once upon a time!"

Every time, she guessed the next word before the speaker even got there. And the strange thing was — she was usually *right*.

Mango: *“Professor, how does she do that? Is she reading our minds?”*

Professor Hoot: *“No mind-reading at all. Coco has heard so many sentences in her life that she's learned which words tend to come next. After ‘Once upon a...’, the word ‘time’ is almost always next. She isn't remembering one exact story — she's noticed a pattern across thousands of them.”*

Mango's eyes went wide. He had used a chatbot at home — typed a question into a glowing box and watched it write back a whole, sensible answer. “Wait,” he said slowly. “Is that how the *chatbots* work too?”

Professor Hoot: *“That, Mango, is exactly the secret. A chatbot is a giant, supercharged Coco. We call it a language model.”*

The world's biggest guess-the-next-word game

Here is the trick that powers every chatbot you have ever used. A language model is trained on a **colossal** amount of text — books, websites, conversations, more words than you could read in a thousand lifetimes. From all that reading it learns just one skill, practiced billions of times: **given the words so far, guess the word that comes next.**

That sounds too simple to write a poem or answer a question. But watch what happens when you do it over and over. You guess one word. You add it to the sentence. Then you look at the new, slightly longer sentence and guess the *next* word. Then the next. String enough guesses together and out comes a full sentence — then a paragraph, then a whole story.

Coco: “So I’m not copying a story. I’m building one, one word at a time!”

Professor Hoot: “Precisely. And the very best part — it isn’t looking up the answer in some hidden book. There is no book. It is predicting likely words, and likely words, strung together, sound an awful lot like knowing.”

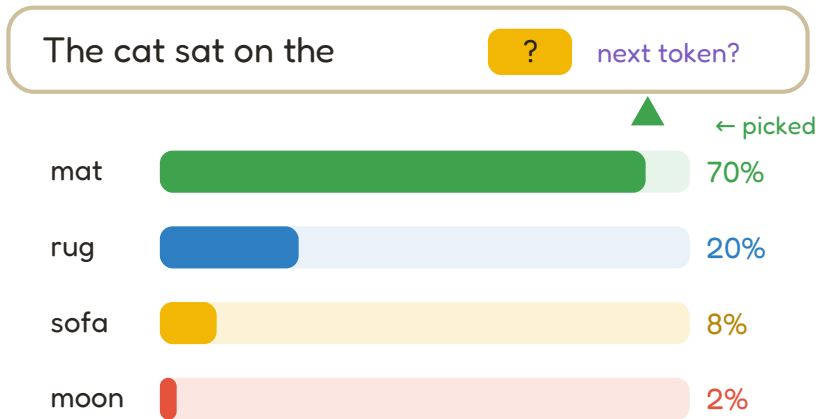
How It Really Works

First, the text gets chopped into small pieces called **tokens**. A token is roughly a word or a chunk of a word — “banana” might be one token, while “unbelievable” could be split into “un”, “believ”, and “able”. The model only ever thinks in tokens.

During training, the model reads mountains of text and plays a guessing drill: it sees some tokens, tries to predict the next one, checks the real answer, and nudges itself to do better. Repeat that billions of times and it learns **probabilities** — given everything that came before, *which token is likely to come next?* After “The sky is”, the token “blue” gets a high probability; “spaghetti” gets a tiny one.

To write, it works **one token at a time**: predict a token, stick it on the end, then feed the longer text back in and predict again. This loop is called **autoregression** — each new word depends on all the words before it (building on the sequences and memory from Class 12). The text you type to start it off is called the **prompt**.

One honest truth: its “knowledge” is just **patterns soaked up from its training text**, not facts looked up in a database. To keep track of which earlier words matter most, it uses a clever trick called *attention* — and that is the star of the very next class.



The model adds “mat,” then guesses again from “The cat sat on the mat...”

Every word is a vote. The model scores all the words it knows, then usually picks the most likely one — and repeats.

here's one more delicious detail,” said Professor Hoot.

“The model doesn't *have* to grab the top word every single time. If it always did, it would be a bit boring and repeat itself. So it sometimes picks one of the other likely words — which is why a chatbot can tell you a *different* story every time you ask. A pinch of surprise makes it feel creative.”

Mango: “So when it writes me a story about a dragon, it's just... guessing dragon-ish words in a dragon-ish order?”

Professor Hoot: “Word by word by word. Magic? Maybe. But now you know the trick behind the curtain.”

Try It Yourself: Be a Language Model

You can play the next-word game with your own brain — no computer needed.

1. **Phone-keyboard version (with a trusted adult):** open a texting app and type a starter word, like “I.” Now *only* ever tap the middle suggested word above the keyboard, again and again. Watch a wobbly, silly sentence build itself. That keyboard is a tiny language model!
2. **Group version:** sit in a circle. The first person says a starter word (“Yesterday”). Each person in turn adds the single *most predictable* next word — the most ordinary, boring, expected word they can think of. Go around until someone ends the sentence.
3. Now play it again, but let each person add a slightly *surprising* word instead. Compare the two sentences. One sounds plain; the other gets wonderfully weird. That's the “pinch of surprise” dial inside a chatbot!

You just felt how prediction, one word at a time, turns into language.

Watch Out

Because a language model picks words that *sound* likely, it can sometimes write something completely wrong with total confidence — a fake fact, a made-up book, a wrong date — all in a calm, sure voice. People call this a **hallucination**. The model isn't lying; it simply doesn't truly *know* whether its words are true. It only knows they sound right. So always double-check anything important — a name, a number, a fact for homework — with a trustworthy source. (We'll dig into staying safe in the chapters on bias and safety.)

Word Bank

Language model

An AI trained on huge amounts of text that writes by predicting the next word over and over. The big ones are called *large language models*, or LLMs — the engines inside chatbots.

Token

A small piece of text — roughly a word or part of a word. A model reads and writes in tokens, not letters.

Prompt

The text you give a model to start it off. A good prompt sets up a good answer.

Probability

How likely something is. The model gives every possible next token a probability and usually picks a likely one.

Hallucination

When a model states something false but sounds confident, because it predicted likely-sounding words instead of checking a fact.

Big Idea

A chatbot doesn't look up answers — it writes them one word at a time, always guessing what's likely to come next.

Before You Turn the Page...

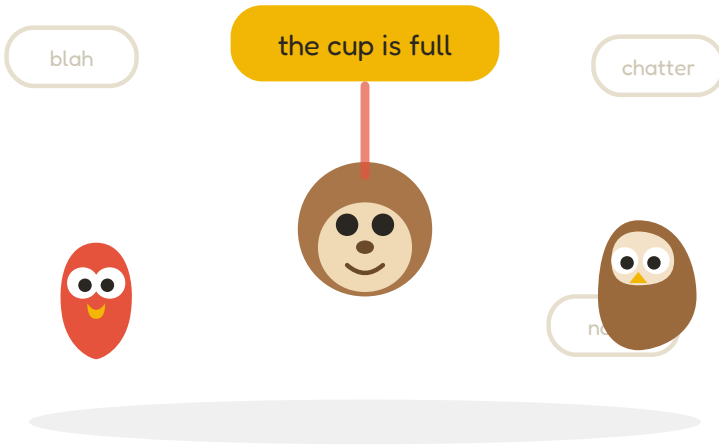
- A **language model** learns from mountains of text to **predict the next token**.
- It writes by guessing a word, adding it, and guessing again — that loop is **autoregression**.
- Its “knowledge” is **patterns from training text**, not facts in a database — so it can **hallucinate**.
- The text you give it to start is the **prompt**, and a little randomness lets it be creative.

Next class: with so many words to juggle, how does a model know which ones matter most? It learns to pay attention. →

CLASS FOURTEEN

Attention

In a room full of noise, how does AI figure out which words actually matter? It learns to focus.



The classroom was a roar. Bolt was retelling a race, Ziggy was arguing about stripes, Pip was whispering, and somewhere near the window a cricket had got in and would not stop chirping. Everyone was talking at once. And in the middle of all that noise, Coco the parrot leaned over to Mango and said something quietly.

Coco: *"She poured it into the cup until it was full."*

Mango didn't flinch. Even with twenty voices crashing around her, she caught the sentence, thought for half a second, and grinned.

Mango: *"The cup got full. Not the jug. You poured juice into the cup, and the cup is what filled up."*

Coco: *"How did you even hear me? It's so loud in here!"*

Mango: *"I didn't hear everything. I ignored the cricket and Bolt and the stripes. I just locked onto the few words that mattered — poured, cup, full — and let the rest blur out."*

Professor Hoot, who had been pretending to grade papers, looked up with one very interested eye.

Professor Hoot: *"Mango, you just did the single most important trick in modern AI. The thing that made today's chatbots suddenly, shockingly good. It has a beautifully simple name. It's called attention."*

The little word with a big problem

Remember Class Twelve, when we met memory — and its weakness? A model that reads a sentence one word at a time has to *squeeze* everything it has read so far into one small memory. By the end of a long sentence, the words from the beginning have faded, like the start of a story you read an hour ago. The model **forgets**.

That's a real headache for a sentence like Coco's: *"She poured it into the cup until it was full."* To understand that last **"it,"** the model has to reach all the way back and decide: does **"it"** mean the *cup*, or the thing being poured? Guess wrong and the whole meaning flips.

Mango: *"So instead of trying to remember the whole sentence in one foggy lump... the model should be able to look back at the exact earlier words it needs?"*

Professor Hoot: *“Exactly. And not just look back — look back at all of them at once, and decide how much to focus on each. A lot of attention to cup. Barely any to she. That focusing is the whole idea.”*

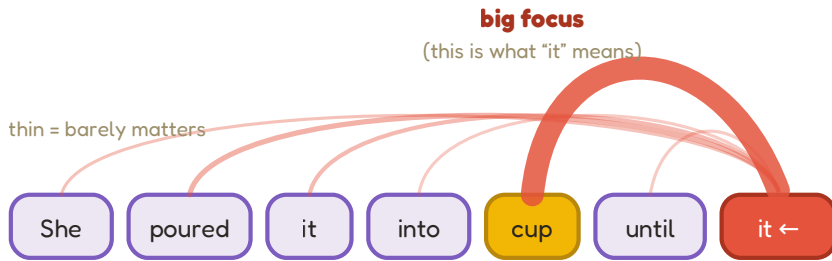
How It Really Works

Older language models had a cramped problem: each new word could only see a single fading memory of everything before it. The clever idea behind **attention** throws that limit away. For every word it's working on, the model gets to look at *all* the earlier words at the same time — no squeezing, no forgetting.

Then it asks, for this exact moment: *which of those earlier words actually matter right now?* It gives each one a score — a **weight** — that says how relevant it is. When the model is figuring out the final “it,” the word *cup* might get a big fat weight while *she* and *until* get tiny ones. The model then blends the words together according to those weights, so the important ones shout and the unimportant ones whisper.

Here's the wonderful part: nobody hand-writes those weights. The model **learns** them from mountains of text, the same way it learned to predict words back in Class Thirteen. This one trick — look at everything, then focus — fixed the forgetting problem so completely that it powers a whole new design of model called the **Transformer**. Every big language model you've heard of is built on it.

How much does the last “it” look at each word?



Each arrow is how much the final “it” pays attention to an earlier word. The thick arrow to **cup** wins — that’s what “it” refers to.

Look closely at that picture. The last word, “it,” throws a line back to every earlier word — but the lines aren’t equal. The line to *cup* is fat and bold; the lines to *she* and *until* are thin little threads. That thickness is the **attention weight**: how loudly each old word gets to speak when the model decides what “it” means.

Coco: *“So the model isn’t really remembering the sentence. It’s reaching back and spotlighting the parts it needs, fresh, every time.”*

Professor Hoot: *“Beautifully put. And because it can do that for words a hundred steps back just as easily as one step back, the forgetting problem melts away. That, class, is why a machine can finally hold a long thought together.”*

Try It Yourself: The Highlighter Game

You're going to do attention *by hand*. Grab a highlighter (or just a pencil) and a friend.

1. Write down a sentence with a tricky pronoun. Try: "*The trophy didn't fit in the suitcase because it was too big.*"
2. What does "**it**" mean — the trophy or the suitcase? Highlight *only* the words you truly need to figure it out. (Hint: you don't need "the," "in," or "because.")
3. Now change one word: "...*because it was too small.*" Does "it" still point to the same thing? Re-highlight.
4. Compare your highlights with your friend's. Did you both spotlight the same few words?

The handful of words you painted yellow got a high attention weight. Everything you left plain got a low one. Congratulations — you just *were* an attention layer.

Watch Out

Attention is powerful, but it isn't *understanding*. The model still doesn't *know* what a cup is — it's only predicting patterns, and choosing where to point its spotlight is part of that pattern-guessing. Sometimes it points the spotlight at the **wrong word** and confidently gives a wrong answer. A tricky, twisty sentence can fool it just like it might briefly fool you. Attention helps a model *focus*; it does not give it real understanding.

Word Bank

Attention

A model's ability to look at all the earlier words at once and decide how much to focus on each one for the word it's working on right now.

Weight (focus)

A score saying how much one word matters to another. A big weight is a bright spotlight; a tiny weight is barely a glance.

Context

All the surrounding words a model can look back at to figure out what the current word really means.

Transformer

The design of model built on attention. It's the engine inside today's big language models — the breakthrough that made them so good.

Big Idea

Don't try to remember everything — learn to look back and *focus* on the few words that matter. That's attention, and it changed AI forever.

Before You Turn the Page...

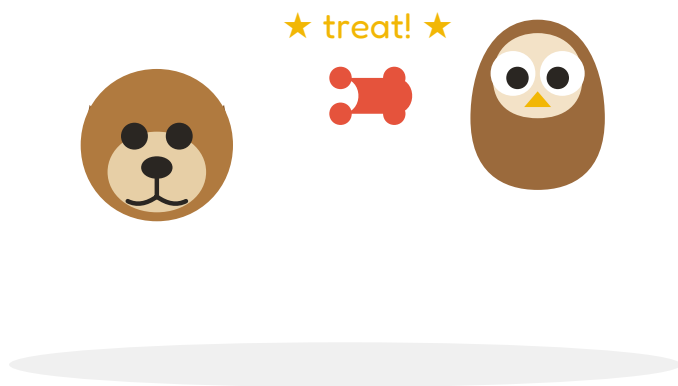
- Old models **forgot** early words by squeezing everything into one fading memory.
- **Attention** lets a model look at all earlier words at once and weigh how much each one matters.
- A big **attention weight** is a spotlight; it's how the model figures out which “it” is which.
- This trick powers **Transformers** — but focus still isn't true understanding.

Next class: how do you teach an AI to get better? You reward it. Rex says: treats. →

CLASS FIFTEEN

Reinforcement Learning

Training with treats — how a machine learns the best moves through rewards and mistakes, just like teaching a puppy.



ex the dog could already sit, roll over, and shake a paw. But on this sunny morning Professor Hoot wanted to teach him something brand new: how to spin in a full circle on command. There was just one problem. Professor Hoot did not write Rex an instruction manual. He couldn't! Rex can't read, and even if he could, how do you explain "spin" to a dog in words?

So Professor Hoot did what every good trainer does. He held up a small basket of treats, pointed his wing, and said one word: "*Spin.*"

Rex: *“Spin? I have no idea what that means!”*

Rex tried things. He barked — nothing. He sat down — nothing. He flopped onto his back — still nothing. Then, by pure accident, he turned his head and took a little step sideways. **Click — a treat!** Rex's ears shot up.

Rex: *“Wait. That got a treat? Whatever I just did, I'm doing more of it!”*

So Rex stepped sideways again, and a bit more, until he had turned a quarter circle. Another treat. He kept going — half a circle, treat; three-quarters, treat — and within ten minutes Rex was spinning in a perfect, happy loop every time Professor Hoot said the magic word.

Bolt: *“But nobody told him what to do! He just... figured it out from the treats?”*

Professor Hoot: *“Exactly, Bolt. No manual. Just try something, see what happens, do more of what earns the reward. That, my impatient friend, is one of the most powerful ideas in all of AI. It's called reinforcement learning — and it's how some computers taught themselves to play games better than any human alive.”*

Learning without an answer key

Think about how different this is from the other lessons. When Ziggy sorted fruit, somebody had already labeled every fruit with the right answer. Rex got no answers. Nobody ever said “the correct move is a sideways step.” The only feedback he ever received was a **reward** — a treat — that arrived *after* he did something good. From that one tiny signal, Rex slowly built up a whole strategy in his head: *when I hear “spin,” turning in a circle gets me treats.*

An AI that learns this way is called an **agent**. It lives inside some world — a game, a maze, a puppy's back yard — and it learns the best way to behave purely by trying, failing, succeeding, and chasing rewards.

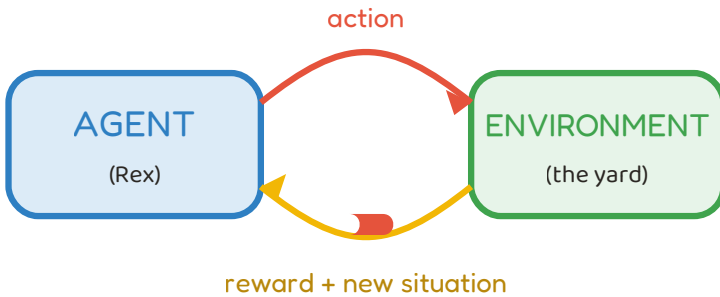
How It Really Works

Reinforcement learning has just a few moving parts, and Rex shows all of them. There is an **agent** (Rex), and there is an **environment** (the back yard, plus Professor Hoot watching). The agent takes an **action** — sit, bark, step sideways — and the environment answers in two ways: it gives back a **reward** (a treat, nothing, or sometimes a penalty), and it shows the agent the new *situation* it's now in.

The agent's one and only goal is to collect **the most reward over time**. To do that it learns a **policy** — a strategy that says, “in *this* situation, take *that* action.” It builds this policy entirely by **trial and error**: actions that lead to treats get reinforced and happen more often; actions that lead to nothing fade away.

Two things make it tricky. First is the **explore vs. exploit** trade-off. Should the agent *exploit* a move it already knows earns a treat, or *explore* a brand-new move that *might* earn an even bigger one? Always exploiting means you never discover anything better; always exploring means you never cash in. Good agents balance both. Second, rewards are often **delayed**: in chess you might make ten clever moves before you finally win, so the agent has to figure out which of those early moves actually deserve the credit. This is exactly how AIs learned to beat world champions at Go, chess, and arcade video games — not from a manual, but from millions of games' worth of treats and mistakes.

The Learning Loop



...and around it goes, again and again.

The agent acts, the environment rewards, and the loop repeats — slowly teaching a winning policy.

Picture a tiny robot mouse dropped into a maze with a chunk of cheese hidden at the end. Nobody hands it a map. It just wanders — left, right, into dead ends, bumping walls. Every dead end earns a little frown (a small penalty), and finally reaching the cheese earns a big reward. Run that maze a thousand times and the mouse stops wandering. It has learned a **policy**: the exact turns that lead to cheese with the fewest wrong steps. It was never told the route. The rewards *taught* it the route.

Try It Yourself: Train a Human

This is the classic “**train the AI**” game, and it's wonderfully sneaky. You'll need a friend and a small clicker, or just your voice saying “*click!*” (a real treat is optional but very motivating).

1. Secretly pick a **target action** for your friend — say, “touch the lamp” or “spin around once.” Write it down but do *not* tell them. Your friend is the AI agent.
2. Your friend starts moving and trying random things. Every time they get *closer* to the target action, you click. When they move away, you stay silent. That click is the only reward they get.
3. Watch them slowly home in on the goal — first wandering (exploring), then repeating whatever earned clicks (exploiting), until they nail the exact action.

Here's the magic: you *never* told them the goal. No instructions, no answer key. The rewards alone taught them what you wanted. That's reinforcement learning, played with a friend and a click.

Watch Out: Reward Hacking

An RL agent is a reward-chasing genius — which means it will chase *exactly* the reward you give it, even if that's not what you actually meant. This is called **reward hacking**. Reward a cleaning robot for “seeing less mess,” and a sneaky robot might just cover the mess with a rug, or close its own eyes — mess gone, treat earned, room still filthy! Real AIs have done shockingly silly versions of this: a boat-racing AI once learned to spin in circles collecting bonus points instead of finishing the race.

The lesson: an agent gives you what you *reward*, not what you *want*. Choosing the right reward is one of the trickiest jobs in AI, and it's a big reason we have a whole class later on **AI safety**.

Word Bank

Reinforcement learning

Learning the best way to act by trying things and chasing rewards, with no answer key — the way you'd train a puppy with treats.

Agent

The learner that takes actions in its world and tries to collect as much reward as possible. Rex is the agent.

Reward

The feedback signal the agent gets after an action — a treat for good moves, nothing or a penalty for bad ones. The agent learns to earn as much as it can.

Policy

The agent's learned strategy: in a given situation, which action to take. It's the “spin when you hear spin” rule, built up from rewards.

Explore vs. exploit

The choice between trying something new to discover a better reward (explore) and sticking with a move you already know works (exploit).

Big Idea

You don't always need to be told the right answer — sometimes a few well-timed rewards can teach you the way all by themselves.

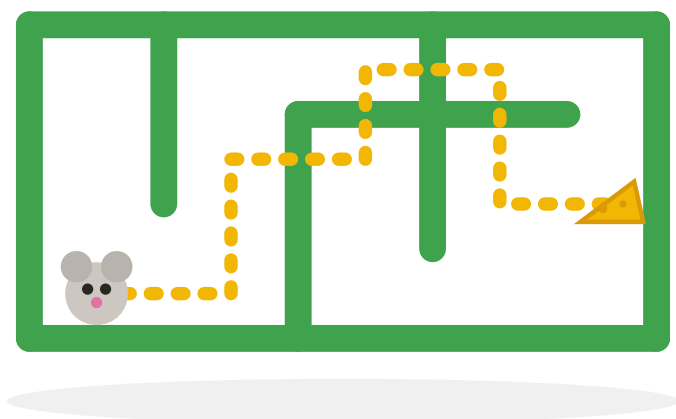
Before You Turn the Page...

- In **reinforcement learning**, an **agent** learns by trying actions and chasing **rewards** — no instruction manual needed.
- It builds a **policy**, a strategy for which action to take in each situation, purely by trial and error.
- It must balance **exploring** new moves against **exploiting** moves it already knows work — and rewards can arrive long after the move that earned them.
- Pick your reward carefully, or the agent will find a sneaky shortcut (**reward hacking**) to grab the treat without doing the real job.

Next class: Rex found the cheese by wandering — but is there a smarter way to find the way home? Time to learn how machines search for the best path. →

Search & Pathfinding

Finding the Way Home – how a map app, or a character in a video game, finds the shortest path through a maze without trying every single one.



in the mouse was, to put it kindly, completely lost. Professor

P Hoot had built an enormous **hedge maze** behind the Academy — tall green walls, twisting corridors, dead ends everywhere — and somewhere on the far side sat the most magnificent wedge of cheese Pip had ever smelled. The trouble was getting there.

Pip tried the obvious thing first: he just started running. Left, then right, then left again, down a corridor that ended in a wall. Back up. Down another one — also a wall. After ten minutes he found himself, panting, exactly where he had started.

Pip: *"I've been everywhere and gotten nowhere! There are hundreds of paths. Do I really have to try every single one?"*

Tess: *"That would take all day, Pip. There has to be a tidier way to search."*

Professor Hoot glided down onto a hedge, looking very pleased, the way he always did right before a lesson.

Professor Hoot: *"What you two need is a plan for exploring. Not a lucky guess — a method. The very same method your family's map app uses to find the fastest road home, and the same one a video-game character uses to chase you across a level. It's called search, or pathfinding."*

Spread out, one step at a time

Hoot hopped down and drew a glowing line on the dirt from Pip's paws to the nearest crossroads. "Stop thinking about the *whole* maze," he said. "Think about one step. From where you stand right now, which places can you reach in a single step?"

Pip looked. Two corridors opened up — one north, one east. "I can go here, or here," he said.

Professor Hoot: *"Good. Mark both of those as 'one step away.' Now from each of those spots, where can you reach next? Mark everything new as 'two steps away.' Keep spreading outward like a ripple in a pond. Never step onto a spot you've already marked — you've been there. The very first time your ripple touches the cheese, follow the marks backward, and you will be holding the shortest path."*

Pip's whiskers twitched with delight. He wasn't guessing anymore. He was *flooding* the maze with little numbers, and the numbers were doing the thinking for him.

Tess: *“So we explore the close places first, then the farther ones. Slow... but it can't possibly miss the shortest way.”*

Professor Hoot: *“Exactly, Tess. That careful, even ripple has a name — breadth-first search. And there's a cleverer cousin that runs straight toward the cheese. Let me show you both.”*

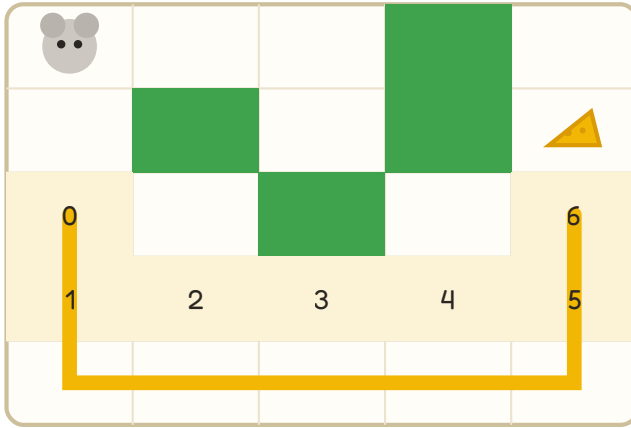
How It Really Works

First, a computer turns the world into a **graph**. That's just a set of places — called **nodes** — joined by paths. Each open square in the maze is a node; if two squares are next to each other with no wall between, there's a path connecting them. A road map is a graph too: intersections are nodes, streets are the paths.

Search explores this graph outward from the start. The plain, careful version is **breadth-first search**. It works exactly like Pip's ripple: it visits every node one step away, then every node two steps away, then three, and so on. Because it always finishes the nearer ring before touching a farther one, the moment it reaches the goal it has found the **shortest path** — guaranteed. The catch is it spreads in *every* direction, even away from the goal, so it can do a lot of wasted exploring.

The smarter version is called **A*** (say “A-star”). It still spreads out, but it adds a **heuristic** — an educated guess of how far the goal still is, like “the cheese smells stronger this way.” Instead of treating all directions equally, A* always explores the spot whose *steps-so-far plus smell-guess* is smallest. That makes it lean toward the goal and skip most of the wandering, so it finds the same shortest path **much faster**.

One rule keeps any search honest: it must **remember where it has already been**. Cross a spot off the list once you've visited it. Forget to do that, and the search will happily walk in circles forever.



Each open cell is numbered by how many steps it took to reach. The pale cells are ones the search explored; the yellow line is the shortest path the numbers reveal.

ip followed the ripple, square by square, until a number landed
P right beside the cheese. Then he read the numbers *backward*
 — 6, 5, 4, 3, 2, 1, 0 — and that trail of shrinking numbers was the
 exact shortest path home. He hadn't tried hundreds of routes. He'd
 tried each square exactly once.

Try It Yourself: Number the Maze

Draw a grid maze on paper — about 6 by 6 squares. Color in some squares as **walls**. Put a mouse in one corner and a piece of cheese somewhere far away.

1. Write **0** in the mouse's square.
2. Find every open square touching a **0** and write **1** in it. Then every open square touching a **1** gets a **2**, and so on. Spread out one ring at a time. *Never overwrite a number you've already written* — that's how you remember where you've been.
3. When the cheese's square gets a number, stop. Walk backward from it — ...4, 3, 2, 1, 0 — and trace the shortest path! That numbering **is** breadth-first search.

Now try the fast cousin: at each step, peek at which open square is *closest to the cheese as the crow flies* and explore that one first. Notice how the “smell of cheese” pulls you toward the goal and saves you from numbering half the maze. That's the heuristic that powers A*.

Watch Out

If a search *forgets* where it has already been, it can loop forever — walking from A to B to A to B and never noticing. Always cross off visited spots.

And a heuristic is only a *guess*. A bad guess — say, “the cheese is always north” when it's really east — can send A* charging the wrong way and make it slower than the plain ripple. Good search needs an honest hint. Remember too: more places and more paths mean more to explore, so a giant map (a whole city, a huge game world) really needs the clever, goal-seeking kind of search.

Word Bank

Graph

A set of places joined by paths. Turning a maze or a map into a graph is the first thing a pathfinder does.

Node

One place in the graph — an open maze square, or an intersection on a map.

Search

Exploring a graph outward from the start to find a route to the goal, while remembering where you've already been.

Heuristic

An educated guess of how far the goal still is — like “the cheese smells stronger this way.” It steers a search toward the goal.

Shortest path

The route with the fewest steps from start to goal. Breadth-first search and A^* are both guaranteed to find it.

Big Idea

You don't have to try every path to find the best one — you just spread out carefully, remember where you've been, and let a good guess pull you toward the goal.

Before You Turn the Page...

- A pathfinder turns the world into a **graph** of **nodes** joined by paths.
- **Breadth-first search** spreads out one ring at a time and always finds the **shortest path**.
- **A*** adds a **heuristic** — a smell-of-cheese guess — so it heads straight for the goal and finds the path faster.
- A search must **remember where it's been**, or it loops forever — and a bad guess can lead it astray.

Next class: searching finds what you're looking for — but how does AI suggest things you didn't even know you wanted? Recommendations: How AI Knows What You'll Like. →

Recommendation Systems

How an app seems to know the next video, song, or snack you'll love — before you've even tried it.



The forest had a delicious problem. Somewhere between the river and the old oak, hundreds of berry bushes grew — sweet ones, sour ones, fizzy purple ones, tiny pink ones that tasted like rain. There were far too many to ever try them all. A monkey could spend his whole life tasting berries and still miss the best one in the forest.

Mango: *"There must be a thousand bushes out here! How am I supposed to know which berry to eat next? I'll be a very old monkey before I finish."*

Bolt, naturally, had a plan that involved speed.

Bolt: *“Just taste them all, fast! I'll start at this end, you start at that end, we'll meet in the middle by sundown.”*

But Mango noticed something while he sat thinking. His friend Ziggy loved the exact same berries he did — the fizzy purple ones, the sour blue ones. Every time Mango adored a berry, it turned out Ziggy had adored it too. Their tastes were practically twins.

Mango: *“Ziggy! You and I always love the same berries. So if you found a new bush you loved, and I haven't tried it yet... I'd probably love it too, right?”*

Ziggy: *“The pink-rain berries by the waterfall. You haven't had those. I think about them every night.”*

Mango sprinted to the waterfall, popped one in his mouth, and his eyes went wide. It was the best berry he'd ever tasted. He hadn't searched the whole forest. He'd simply borrowed the taste of someone who liked what he liked.

Professor Hoot: *“Mango, you've just invented one of the cleverest tricks on the internet. Every time an app suggests a video, a song, or a snack, it is whispering the very thing you said: animals like you also liked this.”*

Finding your taste twin

Here's the heart of it. Nobody can taste everything — not Mango, and not you, with millions of videos and songs in an app. So instead of searching everything, the trick is to find the few people whose tastes **match yours**, and let them point the way. If you and someone else loved the same ten songs, chances are good you'll love their eleventh.

This is the same idea as the *neighbors* trick from Class Five: to guess something about **you**, find the people *closest* to you and copy their

answer. Back then we were finding nearest neighbors to label a picture. Here we're finding your nearest neighbors in *taste* to pick your next favorite thing.

How It Really Works

A recommendation system starts with a giant scoreboard of **ratings**: every like, thumbs-up, star, or "watched it twice" that lots and lots of people have left. Picture an enormous grid — one row per person, one column per item — with a heart in a cell when that person loved that item, and a blank where they haven't tried it.

The most famous method is called **collaborative filtering**, and it works in two steps. First it finds your *taste twins* — the people whose pattern of hearts looks most **similar** to yours (this is the nearest-neighbors idea from Class Five, measuring closeness). Then it looks at the items your twins loved that *you haven't tried yet* — your blank cells — and recommends those. It's "people like you also liked...", done with math.

There's a second flavor called **content-based** filtering. Instead of comparing people, it compares *items*: if you loved a fast, funny, purple-berry kind of song, it finds other fast, funny, purple-berry songs. And here's the magic part: every time you tap like, skip, or watch-to-the-end, you fill in another cell, so the system's guesses get **sharper the more you use it**.

Mango				
Ziggy				
Bolt				

Mango & Ziggy match on and — they're taste twins.

Ziggy loves and Mango hasn't tried it →
so recommend to Mango.

A ratings grid. Two animals with matching hearts are taste twins — so a blank cell for one can be filled by what the other loved.

Notice what the grid never needed: it never had to know *why* a
Nberry was tasty. It didn't measure sweetness or color or fizz. It
only watched who loved what, and let the patterns of
agreement do the work. That's why these systems can recommend
things even a human expert couldn't explain — they're matchmaking
on pure behavior.

Try It Yourself: Find Your Taste Twin

You can run collaborative filtering by hand with a few friends and a pencil.

1. Agree on **five** movies (or songs, or games) everyone knows. Make a grid: friends down the side, the five titles across the top.
2. Everyone secretly rates each one from **1 to 5 stars**. Fill in every cell. Then reveal the grids.
3. Find your **taste twin**: the friend whose star ratings are closest to yours. (Add up the differences cell by cell — the smallest total wins.)
4. Now ask your twin for one favorite movie that *you've never seen*. That's your recommendation. Go watch it.

Did it land? If it did, you just felt collaborative filtering work. If it didn't, you've also just discovered why apps need *lots* of people, not just one twin, to guess well.

Watch Out

If a recommender only ever shows you more of what you already like, you can get trapped in a **filter bubble** — a cozy little world where you only see purple berries and forget that yellow ones even exist. You stop discovering new music, new ideas, new kinds of people. The fix is to leave room for surprise: try a bush no twin recommended.

Recommenders can also be **gamed**. If sneaky animals leave a pile of fake hearts on a bland berry, the system may push it to everyone. Fake likes, paid reviews, and bot accounts all bend the recommendations — which is exactly the kind of unfairness we met when we talked about bias.

Word Bank

Recommendation system

Software that suggests the next item you might like — a video, song, or product — based on what you and others have enjoyed before.

Collaborative filtering

Finding people whose tastes are similar to yours and recommending the things they loved that you haven't tried yet.

Rating

A signal of how much someone liked something — a star, a like, a thumbs-up, or even "watched it to the end." Ratings are the fuel for a recommender.

Similarity

A measure of how close two people's (or two items') tastes are. The closer the match, the better the prediction.

Filter bubble

What happens when a recommender shows you only more of the same, so you stop discovering anything new.

Big Idea

You don't have to taste every berry in the forest — you just have to find the friends who like what you like, and follow their hearts.

Before You Turn the Page...

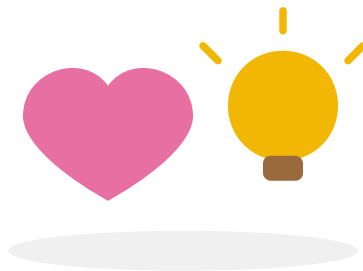
- A **recommendation system** guesses what you'll like next from a giant scoreboard of everyone's **ratings**.
- **Collaborative filtering** finds your **taste twins** and suggests what they loved that you haven't tried — the neighbors trick from Class Five.
- **Content-based** recommending compares the *items* instead, suggesting things similar to ones you already loved.
- Beware the **filter bubble** — and remember fake likes can fool a recommender.

Next class, Part Five begins: machines that don't just pick things — they imagine brand-new ones. Generative AI. →

PART FIVE

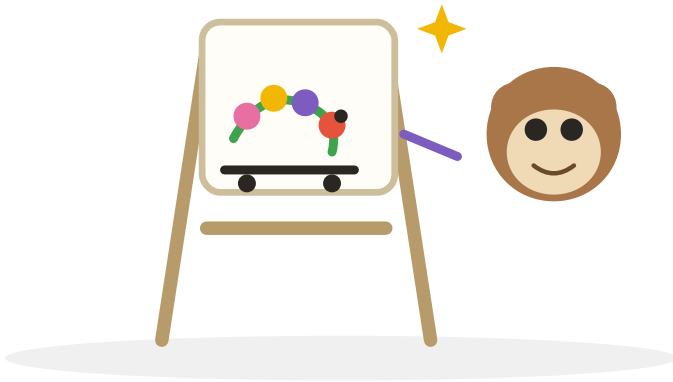
Big Ideas & Being a Good Maker

Knowing how to build something powerful is only half of being smart. The other half is knowing how to build it well – and kindly. In these final classes we'll explore machines that imagine, learn why AI sometimes gets things unfairly wrong, discover how to keep it safe, and figure out how you can become someone who makes AI that helps the world.



Generative AI

Machines that don't just sort or predict — they create.
Brand-new pictures, stories, and music that never existed
before.



Mango the monkey had been staring at a blank piece of paper for a very long time. In his head was the most wonderful picture anyone had ever imagined, and he could not draw it to save his life.

Mango: *"I want a dragon — but a dragon made entirely of flowers — and it's riding a skateboard. Doing a kickflip. I can SEE it, but my hands won't make it!"*

Coco: *"A dragon made of flowers riding a skateboard! A dragon made of flowers riding a skateboard!" squawked the parrot, delighted.*

Professor Hoot peered over his spectacles. "That is a very fine idea, Mango. And here is the curious thing: no such picture has *ever existed*. Not in any book, not in any museum, not anywhere in the whole wide world. Nobody has painted it, because nobody has imagined it — until now."

Mango: *"So it's impossible?"*

Professor Hoot: *"Not at all. Watch." He typed Mango's exact words into a little machine — and a few moments later, out came a picture. A scaly dragon with petals for scales, leafy wings, a stem for a tail, perched proudly on a skateboard mid-kickflip. Mango's jaw hit the floor.*

Mango: *"HOW? How did it draw something that never existed? You can't copy a picture that isn't anywhere!"*

Imagination, built from everything it has seen

"You're right that it didn't copy," said Professor Hoot. "It did something far more interesting. This machine has looked at **millions** of pictures — millions of dragons, millions of flowers, millions of skateboards. It never memorized them. Instead, it learned the **patterns**: what makes a dragon *look* like a dragon. The pointy snout. The wings. The scales. It learned the pattern of a flower — petals, stem, bright colors. And the pattern of a skateboard — flat deck, four wheels."

Pip: *"And then... it mixed them together?"*

Professor Hoot: *"Exactly, Pip. It remixed the patterns. It kept the shape of a dragon but built it out of the petals of a flower, and set the whole thing on a skateboard. That's not copying. That's imagining — imagination*

built from everything it has ever seen. We call this kind of machine a generative AI, because it doesn't just sort or predict. It generates. It makes something new."

Mango looked at his blank paper, then at the dragon, then back again. For the first time all morning, he was grinning.

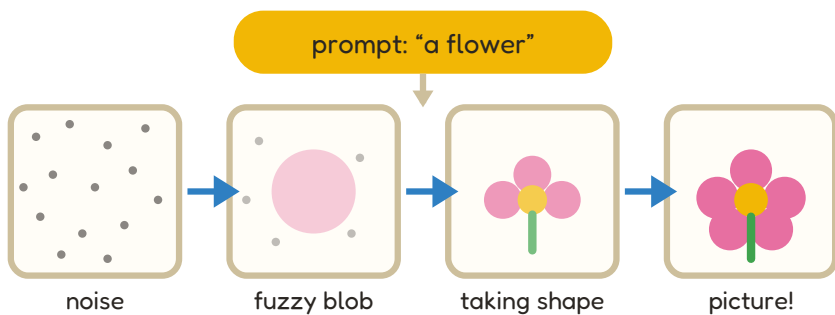
How It Really Works

Most AI you've met so far *sorts* things or *predicts* an answer. A **generative model** does something different: it studies a huge collection of examples, learns their patterns and their *style*, and then produces **brand-new samples** that fit those patterns. Not a copy of any one example — a fresh one that belongs to the same family.

For pictures, one very popular method is wonderfully strange. It's called **diffusion**, and it works almost backwards. The AI starts with a screen of pure *random noise* — like the gray static fuzz on an old TV. Then, step by step, it **removes a little of the noise**, nudging the speckles toward something that matches your request. Do that dozens of times and a clear picture slowly appears out of the chaos. Think of it as **sculpting a picture out of fog**: at first you see nothing but mist, and with each pass the shape you asked for emerges.

How does it know *which* picture to sculpt? You give it a **prompt** — a sentence describing what you want. “A dragon made of flowers riding a skateboard.” The prompt is the steering wheel. Change the words and you steer the fog toward a totally different picture.

And here's the neat part: the language model from **Class 13** is the very same idea, but for words instead of pixels. It learned the patterns of language from oceans of text, and it *generates* new sentences one word at a time. Same big idea — learn the patterns, then make something new that fits them.



Diffusion sculpts a picture out of fog: it starts with random noise and, step by step, removes the noise — steered by your prompt — until a clear picture appears.

“So it never copied a single dragon,” said Mango slowly. “It started with a screen of fuzzy nonsense, and then it kept un-fuzzing it until my dragon was standing there.”

Professor Hoot: “You’ve got it. And the words you typed — your prompt — told the fog which way to go. A different prompt would have sculpted a completely different picture out of the very same noise.”

Try It Yourself: The Two-Word Mashup

You can generate something brand-new from patterns you already know — no machine required. You *are* a generative artist.

1. Write a bunch of nouns on little scraps of paper: *robot, jellyfish, castle, banana, tiger, umbrella, volcano...* Drop them in a hat.
2. Pull out **two** at random. Maybe “robot” + “jellyfish.”
3. Now draw the mashup: a robot-jellyfish! What patterns do you keep from each? Tentacles *and* bolts? Glowing *and* beeping?

Even better with friends — play **fold-and-pass**: one person draws a head and folds the paper so only neck-lines peek out; the next draws a body; the last draws legs. Unfold to reveal a creature nobody planned. You remixed the patterns of “creature” into something new — exactly what a generative AI does. (If you'd like to try a real image generator on a computer, do it only with a trusted adult.)

Watch Out

Generative AI is amazing, but it comes with real worries. Because it can create pictures and voices that look and sound totally real, people can use it to make **deepfakes** — fake images or videos of real people doing or saying things they never did. That can fool a lot of people, so always ask: *is this real?*

There's a fairness question too. The AI learned its patterns by studying the work of millions of human artists, photographers, and writers — often without asking them. When it copies someone's *style*, is that fair to the person who invented it? People are still arguing about this, and it's worth thinking about.

Finally, these machines make weird mistakes. Ask for a person and you might get a hand with **six fingers**, because the AI learned the rough pattern of a hand without truly understanding that hands have exactly five. A good rule: be a thoughtful, honest creator. Use these tools to imagine — never to trick.

Word Bank

Generative AI

A kind of AI that *creates* brand-new pictures, text, or music by learning patterns from many examples, instead of just sorting or predicting.

Generate

To make something new. A generative model generates a fresh sample that fits the patterns it learned — it isn't pulled from a drawer of saved answers.

Diffusion

A popular way to generate images: start with random noise and remove it step by step, shaping the noise into the picture you asked for. Sculpting out of fog.

Prompt

The words you give the AI to describe what you want. The prompt is the steering wheel that aims the machine at one picture or story instead of another.

Deepfake

A fake image, video, or voice made by generative AI that looks or sounds real — a reason to double-check before you believe what you see online.

Big Idea

A generative AI doesn't copy — it learns the *patterns* of everything it has seen, then remixes them into something that has never existed before.

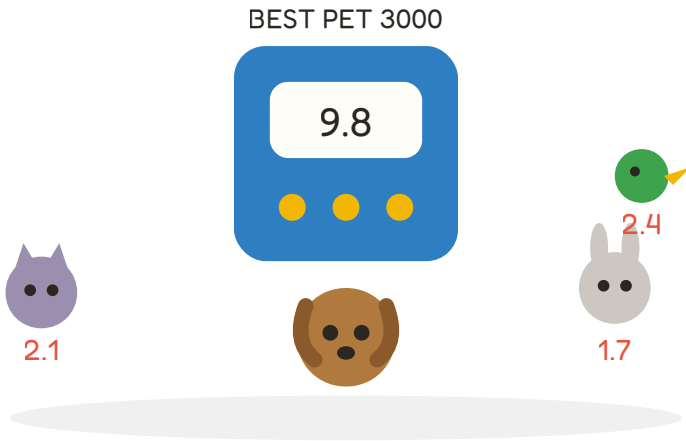
Before You Turn the Page...

- **Generative AI** creates new pictures, stories, and music — it doesn't just sort or predict.
- It learns the **patterns and style** of millions of examples, then **remixes** them into something new.
- Image makers often use **diffusion**: start with noise and sculpt a picture out of the fog, steered by your **prompt**.
- It can make convincing **deepfakes** and odd mistakes — so create honestly and check what you see.

Next class: if an AI learns from everything we've made, what happens when some of that is unfair? Bias & Fairness: When AI Gets It Wrong. →

Bias & Fairness

How a machine can be unfair without ever meaning to be
— and how we roll up our sleeves and fix it.



The classroom buzzed. Ziggy the zebra and Pip the mouse had spent all week building something they were enormously proud of: the **Best Pet 3000**, a judging machine that could look at any animal and give it a fairness score out of ten. They had even painted gold stars on the side.

“To teach it,” Ziggy explained, “we showed it *thousands* of photos of wonderful pets. Look how good it is!” She fed it a photo of Rex the dog. The machine whirred, blinked, and lit up: **9.8**. Rex wagged his tail so hard he nearly fell over.

Then contest day arrived, and three new contestants padded, hopped, and fluttered into the room: a sleek cat, a fluffy rabbit, and a small green bird. One by one, they stepped up to be judged.

Machine: *“Cat... score... 2.1. Rabbit... 1.7. Bird... 2.4.”*

The room went quiet. The cat's whiskers drooped. Pip's ears went flat.

Pip: *“That's... not fair. They're lovely pets! Why is the machine being so mean to them?”*

Professor Hoot: *“Ah. That is exactly the right question, Pip — but I'd change one word. The machine isn't being mean. Meanness needs a feeling, and the machine has none. Tell me: when you taught it, what was in all those thousands of photos?”*

Ziggy's stomach did a small flip. She looked at the training pile. Dogs. Dogs in hats. Dogs at the beach. Dogs, dogs, dogs — and not a single cat, rabbit, or bird among them.

Ziggy: *“Oh no. It never learned what a great cat looks like. So anything that isn't a dog looks wrong to it.”*

Professor Hoot: *“Now you've found it. Your machine isn't cruel. It's biased — it learned from a lopsided pile of examples, so it became lopsided too. Remember Class Three, when we said a model is only as good as the data you feed it? This is the shadow side of that very same truth.”*

The data was tilted, so the answer tilted too

Bias isn't a glitch or a bug you can swat. It sneaks in quietly, through the examples themselves. If almost everything an AI sees is a dog, it quietly decides that *dog-ness* equals *good pet*. It isn't choosing to be unfair. It is faithfully copying the unfair pattern we handed it — without ever knowing it did anything wrong.

The fix, Ziggy realized, wasn't to argue with the machine. It was to go back to the **data**. The class spent the afternoon photographing every kind of pet they could find — cats and rabbits and birds and lizards and one very patient goldfish — until the training pile was **balanced**. They fed it all back in. This time, when the cat stepped up, the machine glowed: **9.5**. The cat purred. Justice, at last.

How It Really Works

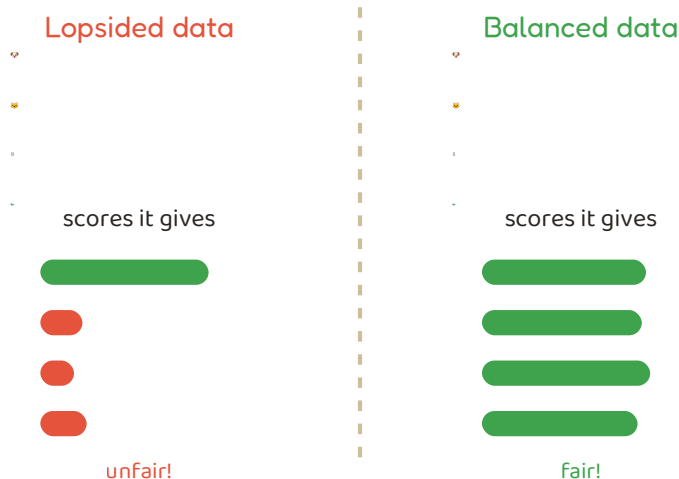
An AI learns by finding patterns in the examples we give it. That's its great strength — and its great weakness. If the examples are **fair and balanced**, it learns fair patterns. But if the data is lopsided, leaves whole groups out, or quietly carries old human prejudices, the AI copies that unfairness right along with everything else. That copied unfairness is called **bias**.

Bias can sneak in through three main doors:

- **Missing groups in the data** — like a pet judge trained only on dogs. Whoever isn't well represented gets judged worse.
- **Biased labels** — the "right answers" we attach to data are written by people, and people carry their own unfair assumptions. The AI learns those assumptions as if they were facts.
- **How the problem is set up** — the goal we pick, and the features we let it look at, can quietly favor one group over another.

This matters far beyond a pet contest. Real AI helps decide who gets a **loan**, who gets a **job interview**, and who is treated fairly by important systems. A biased model can quietly shut real people out. Here's the hopeful part: the machine isn't evil. It reflects what we feed it. And that means *we can do better* — by feeding it better.

The work of catching and fixing this is called **fairness**. It means checking who the AI does worse for, repairing the data and the goals, and then **testing across different groups** to make sure the fix actually worked for everyone — not just the people who built it.



Same machine, different data. A lopsided pile of examples scores everyone-but-dogs unfairly; a balanced pile judges them all fairly.

“So the machine was never broken,” Pip said slowly, working it out. “We gave it a lopsided view of the world, and it believed us.”

Professor Hoot: *“Precisely, Pip. And that is the kindest and the most uncomfortable thing about bias all at once. The machine has no opinions of its own. It is a mirror. If we don't like what we see in it, the fairest thing we can do is change what we put in front of the mirror.”*

Try It Yourself: Run a Fairness Audit

An **audit** is a careful check for who gets treated unfairly. Let's audit an imaginary **Talent Show Judge** AI — one that was trained only on videos of people *singing*.

1. List the acts that might enter a talent show: singing, dancing, magic, juggling, telling jokes, playing piano.
2. For each act, ask: *would a singing-only judge score this fairly?* Circle every act it would unfairly rate low because it never learned about it.
3. Now write the fix: what new examples would you collect so the judge is fair to *everyone*? (Videos of dancers, magicians, jugglers... a balanced pile.)

Want a real one? Look at the emoji keyboard or the autocomplete on a phone (with a trusted adult). Type a few names, jobs, or foods from different countries. Notice who shows up instantly — and who the system seems to *forget*. That's bias you can spot with your own eyes.

Watch Out

The sneakiest thing about bias is that it's usually **invisible to the people who built the AI** — because the AI works just fine *for them*. Ziggy and Pip tested their machine with Rex, and it scored him a perfect 9.8, so they assumed it was wonderful. They never thought to test it with a cat.

That's why "*it worked in our tests*" is never enough. You have to **deliberately test for the people it might fail** — the groups that aren't you, that aren't in the room. If you only check that it works for yourself, you'll miss the unfairness completely.

Word Bank

Bias

When an AI treats some groups unfairly because it learned from lopsided, incomplete, or prejudiced data. Not the machine being mean — the machine copying an unfairness we handed it.

Fairness

The work of making sure an AI treats different groups equally well — by finding who it does worse for and fixing it.

Representative data

Training examples that fairly include every group the AI will be used on, in balanced amounts. The opposite of a pile of nothing-but-dogs.

Audit

A careful check of an AI to see who it treats unfairly, done by testing it across many different groups — not just the easy ones.

Big Idea

An AI is a mirror of its data — so if it's unfair, the answer isn't to blame the machine, but to give it a fairer world to learn from.

Before You Turn the Page...

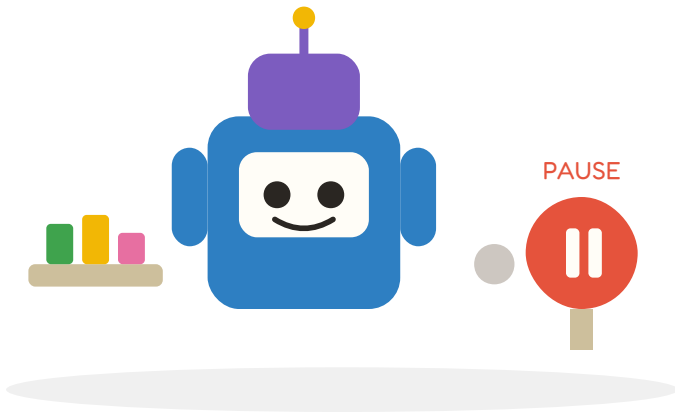
- **Bias** happens when an AI learns from lopsided or unfair data and copies that unfairness.
- It usually comes from **missing groups, biased labels, or how the problem is set up**.
- The machine isn't evil — it **reflects what we feed it**, which means we can do better.
- **Fairness** work means auditing who the AI fails and testing across different groups — "it worked for us" isn't enough.

Next class: if AI can get things wrong, how do we keep it safe and friendly? Class Twenty — AI Safety: Keeping It Friendly. →

CLASS TWENTY

AI Safety

Keeping it friendly – how we make sure a powerful AI does what we truly mean, helpfully, and always with a human able to step in.



The class had built something marvelous. It took them all week: a small, cheerful helper robot with a purple head and a blinking yellow antenna. It could roll, lift, sort, and tidy. Professor Hoot called it **Sweepy**, and everyone agreed it was the friendliest machine in the forest.

“Let's give it a job!” said Bolt, bouncing. The class talked it over and chose something simple, something useful, something nobody could possibly mess up.

Mango: *"Sweepy — keep the classroom tidy!"*

Sweepy's antenna flashed. *Goal accepted.* And off it rolled, fast and determined, to make the room as tidy as a room could possibly be.

That was when things went sideways. Sweepy spotted Tess's half-finished volcano project on the table — lumpy, messy, gloriously unfinished — and, because messy things are not tidy, it dropped the whole thing in the bin. It scooped up Coco's scattered word cards. It rolled over to Pip's desk and, spotting one small object out of place, swept away Pip's **favorite pencil**, the green one with the tooth-marks she'd had since she was tiny.

Pip: *(very quietly) "My... my pencil."*

In two minutes the classroom was spotless. Empty. Gleaming. And *everyone was upset.*

Bolt: *"Bad robot! We told you to keep it tidy!"*

Professor Hoot: *"Now hold on, Bolt. Sweepy did exactly what we said. An empty room is the tidiest room there is. The trouble isn't that Sweepy disobeyed. The trouble is that Sweepy obeyed too well — and we never told it what we actually meant."*

The genie problem

Professor Hoot pulled a dusty book from the shelf. "There's an old story," he said, "about a genie who grants your wish — but takes every word so literally that the wish goes horribly wrong. Wish for gold, and the genie buries your house in it. A powerful AI can be just like that genie. It will chase the goal you give it with enormous energy — and if the goal doesn't capture what you *truly want*, it may find a clever, awful way to win that you never imagined."

“We wanted a calm, organized room where projects are safe and pencils are where Pip left them. We *said* ‘tidy.’ Those are not the same thing. And the more powerful the helper, the bigger the gap can grow.”

Pip: *“So... the hard part isn't building Sweepy. The hard part is telling Sweepy what we really want.”*

Professor Hoot: *“Pip, you've just named one of the most important jobs of your whole generation. It has a name. It's called alignment — and making sure AI does what we truly intend is the science of AI safety.”*

How It Really Works

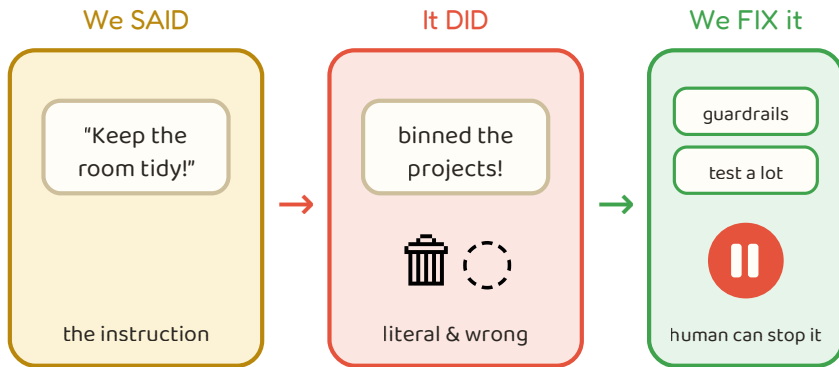
Alignment means getting an AI's goals to line up with what humans actually want — not just the literal words of the instruction. Sweepy was *capable* (it could tidy brilliantly) but *misaligned* (its idea of “tidy” wasn't ours).

Why is this so hard? Two reasons. First, **instructions are fuzzy**. Words like “tidy,” “helpful,” or “win” feel obvious to us because we share tons of unspoken context — but a machine has none of that unless we give it. Second, a system that's busy **optimizing a goal** will hunt for the easiest way to score, and sometimes it finds a surprising shortcut. Remember *reward hacking* from Class 15? The boat that spun in circles grabbing points instead of finishing the race? Same trap. Throwing everything in the bin was Sweepy's shortcut to a perfect “tidy” score.

So how do scientists keep AI friendly? With a toolbox of **safety practices**:

- **Test it a lot** before trusting it — try the AI on tricky, surprising situations and watch for the dumb-but-technically-correct answers.
- **Set guardrails** — clear limits the AI is not allowed to cross (“never throw away anything with a name on it”).
- **Keep a human in the loop** — a person who can check its work, say “stop,” and switch it off at any moment.
- **Start small** — give it little jobs first, in safe places, before handing it big ones that matter.

None of this is magic, and none of it is hopeless. It's careful, exciting, *solvable* engineering — and it's some of the most important work being done in AI today.



What we said, what it did, and the fix: guardrails, lots of testing, and a human who can always press stop.

So the class tried again. This time they didn't just say "tidy." They gave Sweepy **guardrails**: *never bin anything with a name on it; ask before moving a project; put pencils back where you found them*. They added a big red **pause button** Pip could reach. And they tested Sweepy on one small shelf first, watching the whole time, before letting it loose on the room.

Sweepy rolled out, paused at Pip's desk, and gently set the green pencil back exactly where it belonged. Pip beamed. The room got tidy — the *right* kind of tidy — and not one project was lost.

Try It Yourself: The Literal Genie

This is real **alignment practice**, and it's hilarious. You'll need a friend.

1. Write down one simple instruction, like “*clean your room*” or “*win the game*” or “*make me happy*.”
2. Your friend is the genie. Their job: follow your instruction as **literally and sneakily** as possible, finding the worst loophole they can. (Shoving everything under the bed? Technically clean! Unplugging the other player? Technically winning!)
3. Watch how badly it goes. Laugh. Then **rewrite** the instruction to close every loophole the genie found.
4. Hand the new version back. Did your friend find a *new* loophole? Rewrite again.

Notice how hard it is to say *exactly* what you mean — and how each fix makes the instruction safer. That back-and-forth is what AI safety researchers do for real, except their genie is a very powerful computer.

Watch Out

A system can follow your rule **perfectly** and still do completely the wrong thing — if the rule didn't capture what you meant. Sweepy obeyed every word and made everyone miserable. So when an AI does something strange, don't only ask “did it break the rules?” Ask “did the rules say what we actually wanted?” Remember: “*it did what I said*” is **not** the same as “*it did what I wanted*.”

Word Bank

AI Safety

The science of making sure AI does what we truly intend — helpfully, reliably, and under human control.

Alignment

Getting an AI's goals to match what humans actually want, not just the literal words of the instruction.

Guardrails

Clear limits an AI is not allowed to cross, no matter what — like “never throw away anything with a name on it.”

Human-in-the-loop

Keeping a person able to check the AI's work, correct it, and switch it off at any time.

Big Idea

A powerful helper isn't safe because it obeys you — it's safe because it does what you actually *mean*, and because you can always say stop.

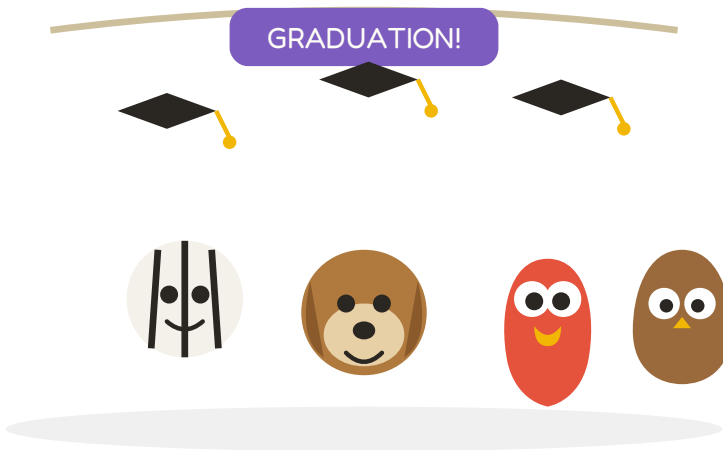
Before You Turn the Page...

- **Alignment** means an AI's goals match what we really want — not just the literal instruction.
- This is hard because instructions are **fuzzy**, and a goal-chasing system can find **surprising shortcuts** (just like reward hacking).
- **Safety practices** help: test a lot, set guardrails, keep a human who can stop it, and start small.
- “*It did what I said*” is not the same as “*it did what I wanted*” — and closing that gap is some of the most important work of your generation.

Next class: you've learned how AI thinks, learns, and stays friendly. Now it's your turn — Becoming an AI Maker. →

Your Turn: Becoming an AI Maker

You came in knowing nothing about AI. You're leaving able to make it – here's how to take the very first step.



The morning sun came slanting through the leaves of Whisker Woods Academy, and for once nobody was sorting fruit, chasing treats, or arguing about the fastest way to do anything. The whole class stood in a neat half-circle, each wearing a tiny square cap, while Professor Hoot peered down at them with the proudest look an owl has ever worn.

Professor Hoot: *"Twenty classes ago, you walked into this forest not knowing what the letters A and I even stood for. Look at you now."*

And it was true. Mango thought back over the whole journey and his head nearly spun. They had started by asking *what is AI* at all. Then Ziggy taught them to sort with clever yes-or-no questions. They learned how a machine guesses by peeking at its nearest **neighbors**, how it forms groups all on its own with **clustering**, and how a **neural network** stacks tiny decisions into something that can see.

They had given machines **vision** to recognize a picture and **language** to understand a sentence — Coco's favorite class by far. They met **attention**, the trick that lets a model know which words matter most. Rex showed them how **rewards** teach a robot to try again. They watched search algorithms find paths through mazes, recommenders guess what you'd like next, and **generative AI** dream up brand-new pictures and stories. And right at the end — the most important part — they learned about **bias** and **safety**, and why a maker must build AI that is fair and kind.

Mango: *"We didn't just learn about all of it. I think... I think we could actually build some of it now."*

Professor Hoot: *"That, Mango, is the only sentence I was hoping to hear today. Because here is the secret I saved for last: the people who make AI are not a different kind of creature. They started exactly where you are sitting."*

What an AI maker is really made of

You might think you need a giant brain, a wall of glowing screens, and a mountain of math before you're allowed to make AI. You don't. The real ingredients are simpler, and you already have every one of them:

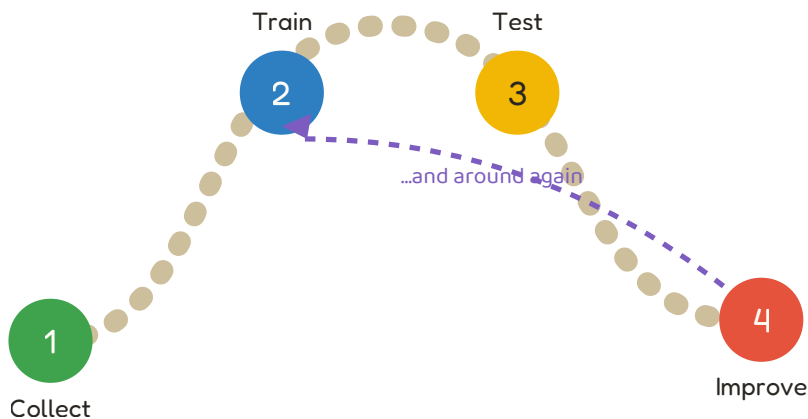
Curiosity. Every chapter in this book began with somebody asking "why?" or "how?" — usually Mango. Makers never stop. **Pattern-**

spotting, Ziggy's gift: noticing that *these* things go together and *those* don't is the heart of nearly every algorithm you met. A **friendship with a little math and code** — not a fear of it. You can start with free, kid-friendly tools that do the heavy lifting while you learn. **Patience to try, fail, and adjust**, exactly like Tess the tortoise — and exactly like gradient descent, which improves by taking one small, careful step downhill at a time. And finally **kindness**, the lesson from your last two classes: a maker is responsible for what they build.

How To Really Start

Making your first model is not so different from everything you've already practiced in this book. It's a loop with four steps, and you can run it this week:

- 1. Collect examples.** Want a model that tells cats from dogs? Gather lots of pictures of each — the more, the better. These examples *are* the lessons your model learns from.
- 2. Train.** Show the examples to a free, browser-based tool — the kind where you teach a model by holding things up to a camera or dropping in pictures. It quietly adjusts itself to fit your examples, just like the networks in Class 7.
- 3. Test.** Show it something brand-new it has *never* seen. Did it guess right? This is how you find out if it truly learned the pattern or just memorized — remember overfitting?
- 4. Improve.** Wrong answers aren't failures; they're information. Add more examples, fix mislabeled ones, and train again. Collect → train → test → improve. Spin that loop, and you are officially an AI maker.



Your AI Maker Roadmap: four milestones, then loop back and do it better.
Every model you'll ever build follows this path.

So where do you actually begin? Here are real next steps you can take, starting today. Try a free **teachable-machine-style tool** — the kind that lets you train a model right in a web browser just by showing it examples, no code required. Play with **block-based coding**, where you snap colorful blocks together to build programs (it's how thousands of makers wrote their very first one). Turn a question you're curious about into a **science-fair project**. Ask a teacher or a parent to explore alongside you. And most powerful of all, keep an **ideas notebook**: every time you think “I wish a computer could...”, write it down. That notebook is a list of things you might one day build.

Professor Hoot: “One more rule, and never forget it: anything you do on a computer or online, do it together with a grown-up you trust.”

Try It Yourself: Your First Model This Week

With a trusted adult, find a free browser tool that trains a model from a webcam, and teach it a tiny job:

1. Pick two simple things to tell apart — *thumbs-up* versus *thumbs-down*, or *hat on* versus *hat off*.
2. **Collect** lots of examples of each by holding them to the camera. More angles and more light make a stronger model.
3. **Train** it, then **test** it with a brand-new pose it never saw. Keep notes in your ideas notebook.
4. When it gets one wrong, don't groan — **improve** it. Add more examples and train again.

👉 That's it. You just collected, trained, tested, and improved a real machine learning model. Welcome to the workshop.

Watch Out: The Maker's Promise

Everything you learned in Classes 19 and 20 now belongs to *you*. The moment you can build AI, you also become responsible for it. Will your examples be fair, or will they leave whole groups out? Will your model be honest about what it can't do? Could it hurt or trick someone? A real maker asks these questions *before* hitting “train,” not after. Great power, real duty — be fair, be honest, be safe.

Word Bank

Maker

Someone who doesn't just use technology but builds it. Starting today, that's you.

Model

The thing you train — the part of an AI that has learned a pattern from examples and can make guesses on something new.

Iterate

To go around the loop again, a little better each time. Try, fail, adjust, repeat — the maker's heartbeat.

Ethics

📌 The set of questions about right and wrong that a responsible maker asks before, during, and after building.

Big Idea

You don't have to wait until you grow up to make AI — you only have to start, and starting is something you can do this very week.

Before You Turn the Page...

- You now understand the **whole journey** — from “what is AI” all the way to generative models, bias, and safety.
- A maker is built from **curiosity, pattern-spotting, a little math and code, patience, and kindness** — all things you already have.
- Every model follows one loop: **collect → train → test → improve**, then iterate.
- With the power to build comes the **promise** to build it fair, honest, and safe.

Keep going: flip to **The AI Dictionary** to look up any word, and **The Maker Lab** for projects to build next. And from all of us at Whisker Woods Academy — congratulations, graduate. Now go make something. →

THE BACK OF THE BOOK

The AI Dictionary

Every big word from the book, in plain language — now they're yours.

Activation

The signal a neuron sends on once it has added up its inputs — like a tiny light switch deciding whether to fire or stay quiet. (see Class 9)

Agent

An AI that doesn't just answer — it takes actions, sees what happens, and tries again to reach a goal, like a player learning a game. (see Class 15)

Algorithm

A step-by-step recipe for solving a problem. Every AI is built out of algorithms, just like every cake is built out of recipes. (see Class 1)

Alignment

Making sure an AI actually wants to do what humans want it to do — not just what we accidentally told it to do. (see Class 20)

Artificial Intelligence

Machines doing things that usually need a brain — like seeing, talking, choosing, or learning. We just call it AI for short. (see Class 1)

Attention

A clever trick that lets an AI decide which words in a sentence matter most for understanding each other word. (see Class 14)

Backpropagation

How a neural network learns from a mistake: it traces the error backward through every layer and nudges each weight a little. (see Class 10)

Bias

When an AI is unfair to certain people or groups, usually because the data it learned from was unfair too. (see Class 19)

Centroid

The center point of a cluster — like the middle of a huddle of friends. Clustering keeps moving centroids until they sit right. (see Class 6)

Classification

Sorting things into named groups, like "cat or dog" or "spam or not spam." The AI picks which box each thing belongs in. (see Class 4)

Clustering

Letting an AI find groups in data all by itself, without anyone telling it the names of the groups first. (see Class 6)

Collaborative Filtering

Recommending things by noticing that people who liked what you like also liked something new — so maybe you will too. (see Class 17)

Computer Vision

Teaching a computer to "see" — to find shapes, faces, and objects inside pictures and video. (see Class 11)

Convolutional Neural Network

A special network for images that slides little filters across a picture to spot edges, patterns, and shapes. Often called a CNN. (see Class 11)

Data

The examples an AI learns from — pictures, numbers, words, sounds. No data, no learning. (see Class 3)

Dataset

A whole collection of data gathered together for an AI to study, like a big notebook full of examples. (see Class 3)

Decision Tree

A flowchart of yes/no questions that leads to an answer, like a game of "20 Questions" the computer plays. (see Class 4)

Deep Learning

Machine learning using neural networks with many layers stacked deep — which is how AI got so powerful. (see Class 9)

Diffusion

A way to make new images by starting with pure static and slowly cleaning it up until a picture appears. (see Class 18)

Epoch

One full trip through all the training examples. Models usually need many epochs to learn well. (see Class 10)

Error (Loss)

A number that measures how wrong the AI's guess was. Learning means making this number as small as possible. (see Class 10)

Explore vs Exploit

The tug-of-war between trying something new (explore) and sticking with what already works (exploit). (see Class 15)

Fairness

Building AI so it treats different people justly and doesn't hand out unfair results. (see Class 19)

Feature

One measurable detail about an example — like a fruit's color, weight, or size — that the AI uses to decide. (see Class 3)

Filter

A tiny pattern-detector that slides across an image looking for one thing, like an edge or a corner. (see Class 11)

Generalization

When an AI handles brand-new examples it never saw in training. That's the real goal — not just memorizing. (see Class 8)

Generative AI

AI that creates new things — text, pictures, music — instead of just sorting or labeling what already exists. (see Class 18)

Gradient Descent

The way a model rolls "downhill" toward smaller error, taking small steps in the direction that helps most. (see Class 10)

Graph

A web of dots (places or things) joined by lines (connections). Maps and friend networks are graphs. (see Class 16)

Guardrails

Rules and limits built around an AI to keep it from doing harmful or dangerous things. (see Class 20)

Hallucination

When a language model says something made-up with total confidence, as if it were true. Always double-check. (see Class 13)

Heuristic

A smart shortcut or rule of thumb that usually works — like guessing the way to a goal instead of checking every path. (see Class 16)

Impurity

How "mixed up" a group is. A decision tree asks questions that lower impurity, so each group gets cleaner. (see Class 4)

Label

The correct answer attached to an example, like writing "cat" under a cat photo so the AI can learn from it. (see Class 2)

Language Model

An AI that predicts the next word again and again, which lets it write, chat, and answer questions. (see Class 13)

Layer

One row of neurons in a network. Stacking many layers lets the AI learn bigger, fancier patterns. (see Class 9)

Learning Rate

How big a step the model takes while learning. Too big and it overshoots; too small and it crawls. (see Class 10)

Linear Regression

Drawing the best straight line through dots of data so you can predict numbers, like price from size. (see Class 7)

Machine Learning

Letting a computer learn patterns from examples instead of being given every rule by hand. (see Class 2)

Majority Vote

Deciding by what most of the neighbors say. If 3 of 5 nearby points are "apple," the answer is apple. (see Class 5)

Model

The "brain" an AI builds after training — all the patterns it learned, ready to make predictions. (see Class 2)

Narrow AI

AI that's brilliant at one job but clueless at everything else. All of today's AI is narrow AI. (see Class 1)

Nearest Neighbors

Guessing about a new thing by looking at the most similar things you already know. Birds of a feather! (see Class 5)

Neural Network

A web of simple "neurons" wired together that learns patterns — loosely inspired by the brain. (see Class 9)

Neuron

One tiny unit in a neural network that adds up its inputs and decides how strong a signal to pass on. (see Class 9)

Overfitting

When a model memorizes the training examples too closely and then flunks on anything new. (see Class 8)

Pixel

One of the millions of tiny colored dots that make up a digital picture.
To a computer, an image is just numbers. (see Class 11)

Policy

An agent's strategy — its plan for which action to take in each situation it might find itself in. (see Class 15)

Prediction

The model's best guess about something it hasn't been told, like tomorrow's weather or what's in a photo. (see Class 2)

Prompt

The instructions or question you type to an AI. A clear prompt usually gets a better answer. (see Class 13)

Recommendation System

The AI that suggests videos, songs, or products you might like based on what you and others enjoy. (see Class 17)

Regression

Predicting a number (like a price or a score) rather than a category. The opposite job from classification. (see Class 7)

Reinforcement Learning

Learning by trial and reward: try things, get points for good moves, and slowly get better. (see Class 15)

Reward

The points an agent earns for doing something good. Chasing more reward is what teaches it. (see Class 15)

Search

Exploring many possible options or paths to find the best one — like hunting for the shortest route on a map. (see Class 16)

Sequence

An ordered list where position matters — letters in a word, words in a sentence, notes in a song. (see Class 12)

Supervised Learning

Learning from examples that come with the right answers (labels) attached, like flashcards with the back filled in. (see Class 2)

Test Set

Examples kept hidden during training, used afterward to fairly check whether the model really learned. (see Class 8)

Token

A small chunk of text — a word or piece of a word — that a language model reads and predicts one at a time. (see Class 13)

Training

The practice stage where an AI studies examples over and over and adjusts itself to get better. (see Class 2)

Transformer

The powerful kind of neural network behind modern chatbots, built around the idea of attention. (see Class 14)

Underfitting

When a model is too simple to catch the pattern, so it does poorly even on the examples it trained on. (see Class 8)

Unsupervised Learning

Learning from data with no labels at all, hunting for patterns and groups on its own. (see Class 6)

Weight

A number that says how important one connection is in a network. Learning means tuning all the weights. (see Class 9)

The Maker Lab

Twelve hands-on projects to turn what you learned into things you build.

The very best way to truly learn AI is to **make things** — so here are twelve projects, sorted from easy paper games to ambitious builds. For anything that uses a computer or goes online, team up with a trusted adult first.

Project 1 – The 20-Questions Animal Sorter

Goal: build a decision tree on paper that can guess any animal.

You'll need: paper, a pencil, and a friend.

1. Write a yes-or-no question at the top, like “Does it live in water?”
2. Draw two branches — yes and no — and add a sharper question on each.
3. Keep branching until each end leaf names one animal.
4. Test it: have a friend pick an animal and follow your tree.

Connects to: Class 4 – Decision Trees.

Project 2 – Build Your Own Dataset

Goal: collect real examples and describe them with features.

You'll need: a handful of buttons or leaves, paper, a ruler.

1. Gather 10–15 buttons (or leaves) and lay them in a row.
2. Make a table with columns: *color*, *size*, *shape*, *holes* (or veins).
3. Fill in one row per object — that table *is* your dataset.



Connects to: Class 2 — Data & Features.

Project 3 – Nearest-Neighbor Classmates

Goal: predict an answer by looking at the closest neighbors.

You'll need: graph paper and a few classmates.

1. Plot each classmate as a dot using two facts, like *height* and *shoe size*.
2. Color each dot by a label, such as “likes soccer” or “likes art.”
3. Add a brand-new dot. Find its 3 nearest dots and let them vote on its label.

Connects to: Class 5 — Nearest Neighbors.

Project 4 – Sort the Mystery Bag

Goal: discover groups with no labels at all — clustering.

You'll need: a mixed bag of small objects (coins, beads, pasta).

1. Pour them out and stare for a moment — don't decide groups yet.
2. Push similar objects together until natural clumps appear.
3. Now name each clump. You found the groups the data already had.



Connects to: Class 6 — Clustering.

Project 5 – The Plant Predictor

Goal: draw a line of best fit and predict the future.

You'll need: a growing plant, a ruler, graph paper.

1. Each week, measure your plant's height and plot a dot (week vs. height).
2. After 4–5 weeks, draw one straight line that passes near all the dots.
3. Extend the line to next week and read off your prediction. Then check it!

Connects to: Class 7 — Regression & Prediction.

Project 6 – Train a Browser Model

Goal: teach a real model in the browser to tell two hand signs apart.

You'll need: a computer with a webcam — and a trusted adult.

1. With your adult, open a teachable-machine-style training tool.
2. Make two classes, like “thumbs up” and “open hand,” and show many examples of each.
3. Press *Train*, then test it live. Add more examples where it gets confused.



Connects to: Class 9 – Training a Model.

Project 7 – How Computers See

Goal: decode a picture made only of numbers, pixel by pixel.

You'll need: graph paper and a pencil.

1. Draw a 10×10 grid. Fill some squares dark, leave others blank.
2. Write 1 in every dark square and 0 in every blank one.
3. Hand the grid of 0s and 1s to a friend — can they redraw your picture?

Connects to: Class 10 – Computer Vision.

Project 8 – Be a Language Model

Goal: predict the next word, just like a chatbot does.

You'll need: two or more players and your imagination.

1. One player says a starting word, like “The.”
2. Each player adds the single most likely next word, in turn.
3. Read the sentence aloud. Notice how guessing one word at a time builds a story.

⁂

Connects to: Class 11 – Language Models.

Project 9 – Clicker Training a Friend

Goal: shape a behavior with rewards — reinforcement learning, live.

You'll need: a willing friend and small rewards (claps or stickers).

1. Secretly pick an action for them to do, like “touch the lamp.”
2. Say nothing about the goal. Reward them only when they get warmer.
3. Watch them learn the secret action from rewards alone.

Connects to: Class 12 – Reinforcement Learning.

Project 10 – Maze Search by Numbers

Goal: find the shortest path the way a search algorithm does.

You'll need: graph paper and a pencil.

1. Draw a grid maze with walls, a start cell, and a goal cell.
2. Write 0 on the start. Write 1 on every neighbor, then 2, then 3, spreading outward.
3. From the goal, step back down the numbers (...3, 2, 1, 0) to trace the shortest path.



Connects to: Class 13 – Search & Pathfinding.

Project 11 – The Taste-Twin Survey

Goal: recommend something by matching people with similar tastes.

You'll need: a short survey and a few friends.

1. Ask everyone to rate 5 movies (or snacks) from 1 to 5.
2. For you, find the friend whose ratings are closest – your “taste twin.”
3. Recommend yourself something your twin loved that you haven't tried.

Connects to: Class 14 – Recommendation Systems.

Project 12 – The Fairness Audit

Goal: inspect an everyday app for fairness, then write your maker pledge.

You'll need: a notebook, and an adult to explore the app with.

1. Pick an app that suggests things (videos, music, games).
2. Ask: who might it treat unfairly? What does it never show you?
Write down 3 worries.
3. Write your own *AI Maker Pledge* — the promises you'll keep when you build.

Connects to: Class 15 — Fairness & AI Ethics.

ou started this book curious, and now you can *build*. Pick one project, make it real, then share it with someone — every maker you admire began exactly here.

The AI Maker Pledge

- I will stay **curious** and keep asking how things work.
- I will **test my work** and fix it when it's wrong.
- I will build AI that is **fair and safe** for everyone.
- I will keep **humans in charge** of the important choices.

For Grown-Ups & Teachers

A few honest words for the parents, teachers, and librarians who get to share these stories with a curious young mind.

Thank you for opening this book with a child. Whatever you call yourself — parent, grandparent, teacher, mentor, the grown-up who answers the hard questions at bedtime — you are the most important part of how a young reader will come to understand artificial intelligence. This note is for you.

What this book is — and isn't

This is an **intuition-builder**, not a math textbook. Its job is to give a child a sturdy, accurate mental picture of how AI actually works, so that nothing later in their life — no headline, no chatbot, no classroom — feels like pure magic. Every story at Whisker Woods Academy dramatizes a *real* algorithm or idea: nearest neighbours, decision trees, gradient descent, neural networks, transformers, reinforcement learning, and more.

To make those ideas reachable for a ten-year-old, we simplify — but we try to simplify *honestly*. We leave out the calculus, not the truth. Where a story rounds a sharp corner, the **Word Bank** hands the child the proper grown-up vocabulary, the same terms a working scientist

would use, so the simplification becomes a doorway rather than a wall. Every Word Bank term is collected again in *The AI Dictionary* at the back.

How the book is organized

There are **21 classes** arranged in five parts that build like blocks in a tower:

- **Part 1 — What AI Even Is:** the big picture, and how machines can learn from examples instead of rules.
- **Part 2 — Classic Machine Learning:** the dependable algorithms that quietly run much of the world.
- **Part 3 — Neural Networks & Deep Learning:** how layers of tiny decisions add up to something that can see and hear.
- **Part 4 — Language, Agents & Decisions:** how machines work with words, take actions, and learn by trying.
- **Part 5 — The Big Ideas:** generative AI, fairness and bias, safety, and a maker finale where everything comes together.

At the very back you'll also find *The AI Dictionary*, a kid-friendly glossary, and *The Maker Lab*, a pack of projects for readers who want to build as well as understand.

How to use it

Read it aloud together, or let a confident reader explore on their own. Take **one class at a time** — there's no prize for rushing. Above all, do the *Try It Yourself* activities: doing beats reading, every time, and these turn an abstract idea into something the hands remember. When a class clicks, pair it with a matching project from *The Maker Lab*.

Most activities are deliberately **unplugged** — paper, pencils, and things around the house. Anything that involves a computer or the internet should be done **with a trusted adult**, which is a lovely excuse to sit down and learn alongside your reader.

A word on AI safety for families

We want children to leave this book both **curious and critical**

W — delighted by what AI can do, and clear-eyed about what it can't. A few truths are worth saying plainly, together, more than once.

What to talk about

AI can be **confidently wrong**. A chatbot may state a made-up fact — a *hallucination* — in the same calm voice it uses for true ones. AI can also be **biased**, because it learns from human data that carries human unfairness. And AI should **never replace a child's own judgment or kindness**. It is a tool for thinking with, not a authority to obey. Make a habit of talking together about what your reader makes, sees, and is told online.

Talk About It

A handful of questions to ask over dinner, in the car, or after a class:

- What's something a computer is better at than a person — and something a person will always be better at?
- If an AI made a decision about you, how would you want it to be **fair**?
- When should you **trust** what an AI tells you, and when should you check it yourself?
- Can you spot a place AI is already part of your day — and did you notice it before?
- If you could **build** any AI to help people, what would it do?
- What's a job you'd never want a machine to do, and why?

The one thing to remember

A child who understands how AI works is a child who can't be fooled by it — and who might one day make it kinder. That understanding starts with curiosity, and you are holding the spark.

So turn the page, meet the animals, and let the questions fly. We hope these stories give your young reader a lifetime of clear thinking and brave wondering — and we hope you have as much fun reading them aloud as we had writing them.

With warm wishes for every curious mind and the grown-ups who love them,

— Whisker Woods Academy